Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Copy Constructor, Shallow and Deep Copies, Σύνθεση Αντικειμένων

Παναγιώτης Ηλία

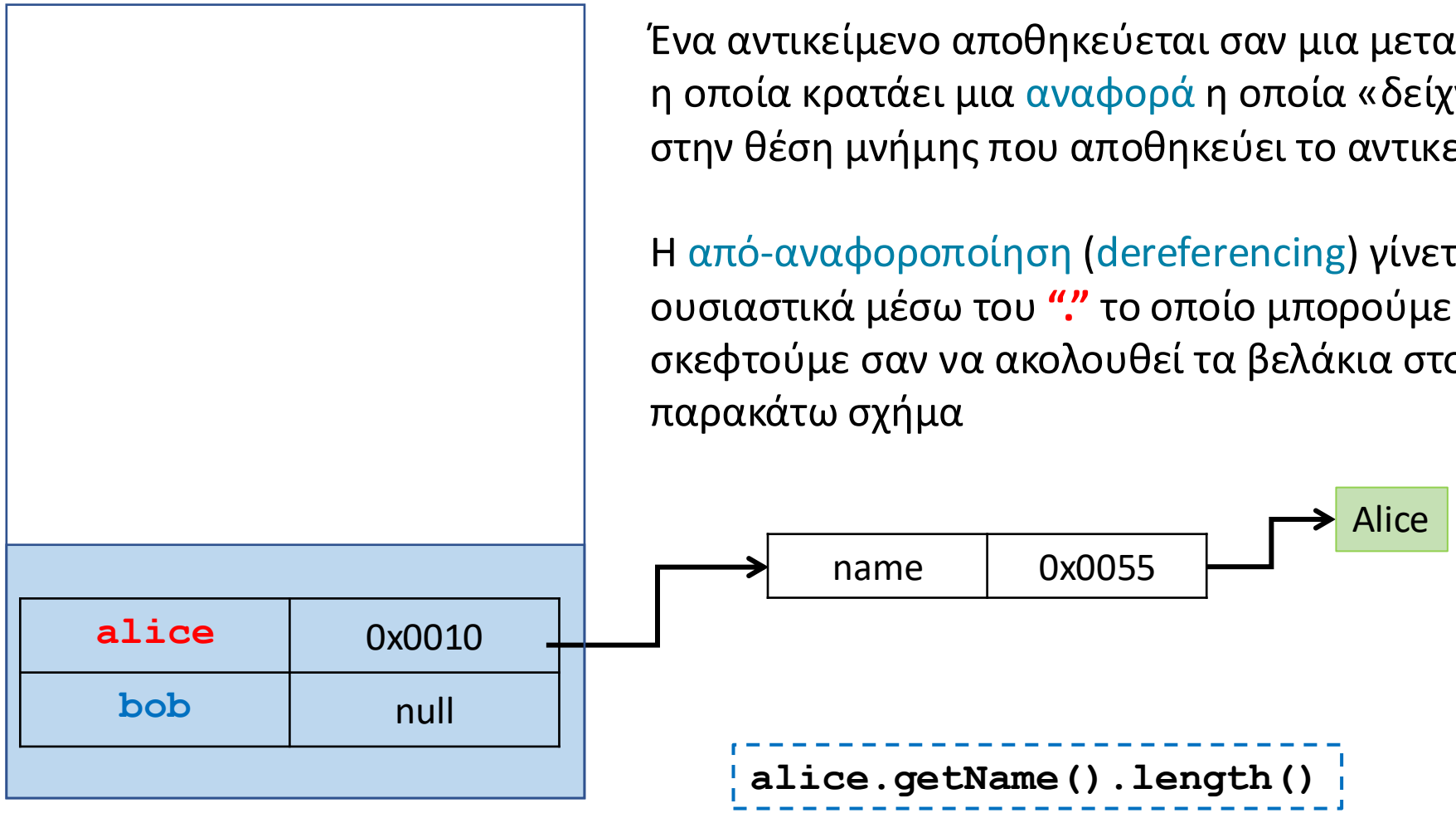
```
class Person
{
    private String name;

    public Person(String name){
        this.name = name;
    }

    public String getName(){
        return name;
   }
}
```

```
class PersonTest
{
    public static void main(String args[])
    {
         Person alice = new Person("Alice");
         Person bob;
         System.out.println(alice.getName());
         System.out.println(alice.getName().length());
    }
}
```

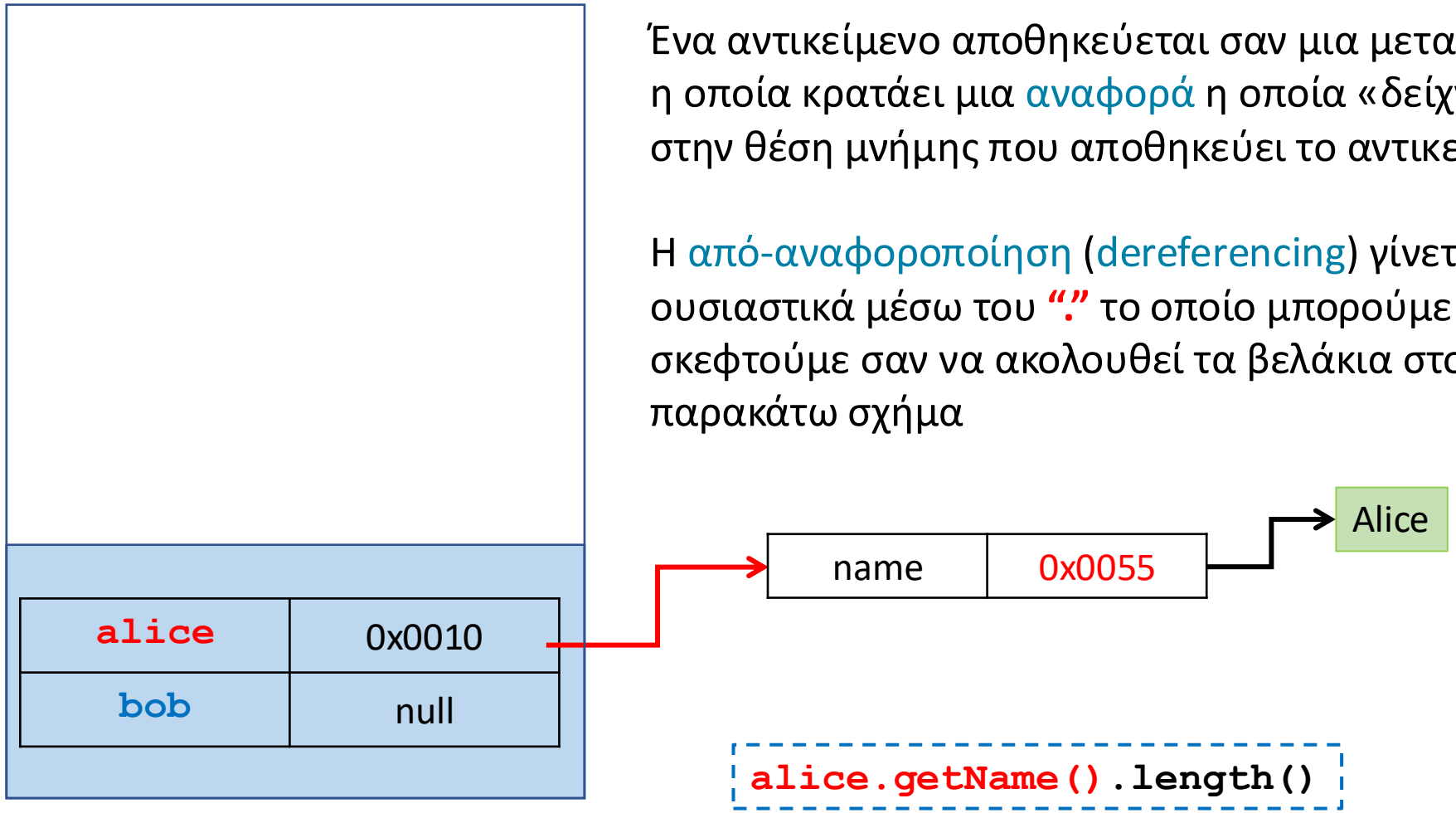
Dereferencing



Ένα αντικείμενο αποθηκεύεται σαν μια μεταβλητή η οποία κρατάει μια αναφορά η οποία «δείχνει» στην θέση μνήμης που αποθηκεύει το αντικείμενο.

Η από-αναφοροποίηση (dereferencing) γίνεται ουσιαστικά μέσω του “.” το οποίο μπορούμε να σκεφτούμε σαν να ακολουθεί τα βελάκια στο παρακάτω σχήμα

Dereferencing

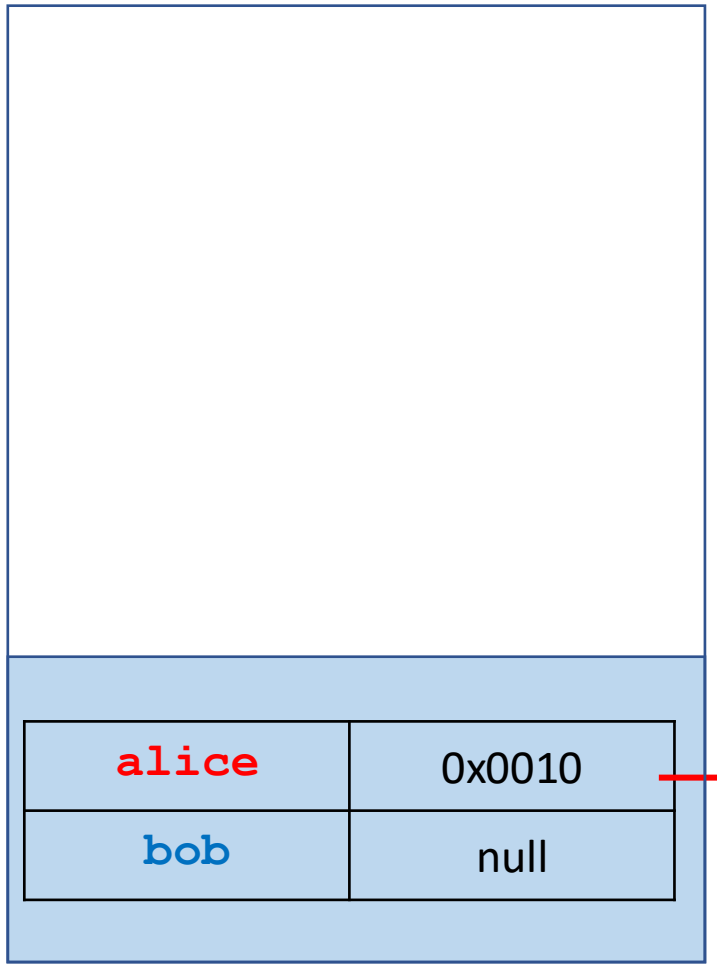


Ένα αντικείμενο αποθηκεύεται σαν μια μεταβλητή η οποία κρατάει μια αναφορά η οποία «δείχνει» στην θέση μνήμης που αποθηκεύει το αντικείμενο.

Η από-αναφοροποίηση (dereferencing) γίνεται ουσιαστικά μέσω του **“.”** το οποίο μπορούμε να σκεφτούμε σαν να ακολουθεί τα βελάκια στο παρακάτω σχήμα

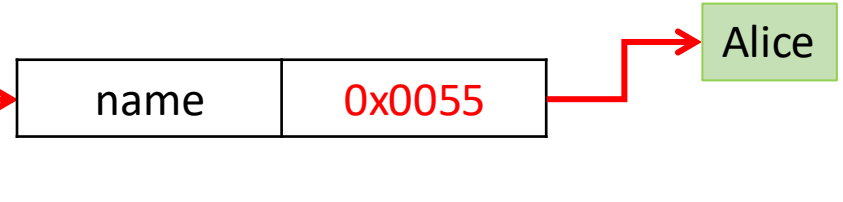
```
alice.getName().length()
```

Dereferencing



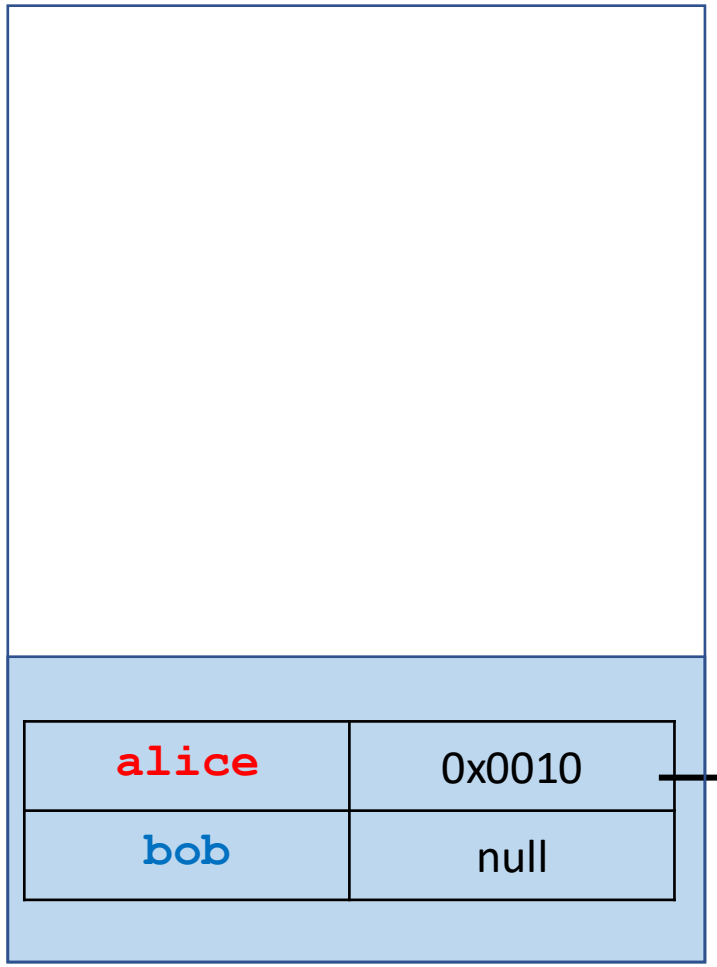
Ένα αντικείμενο αποθηκεύεται σαν μια μεταβλητή η οποία κρατάει μια αναφορά η οποία «δείχνει» στην θέση μνήμης που αποθηκεύει το αντικείμενο.

Η από-αναφοροποίηση (dereferencing) γίνεται ουσιαστικά μέσω του **“.”** το οποίο μπορούμε να σκεφτούμε σαν να ακολουθεί τα βελάκια στο παρακάτω σχήμα



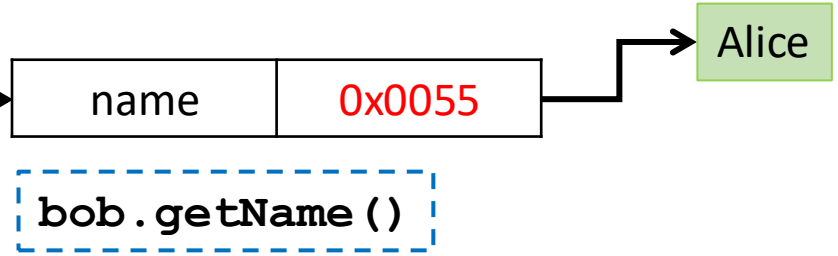

```
alice.getName().length()
```

Dereferencing



Ένα αντικείμενο αποθηκεύεται σαν μια μεταβλητή η οποία κρατάει μια αναφορά η οποία «δείχνει» στην θέση μνήμης που αποθηκεύει το αντικείμενο.

Η από-αναφοροποίηση (dereferencing) γίνεται ουσιαστικά μέσω του **“.”** το οποίο μπορούμε να σκεφτούμε σαν να ακολουθεί τα βελάκια στο παρακάτω σχήμα



Στην περίπτωση αυτή θα πάρουμε λάθος (είτε run-time, είτε compile error) γιατί δεν υπάρχει διεύθυνση να ακολουθήσουμε

```
class Person
{
    private String name;

    public Person(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }
}
```

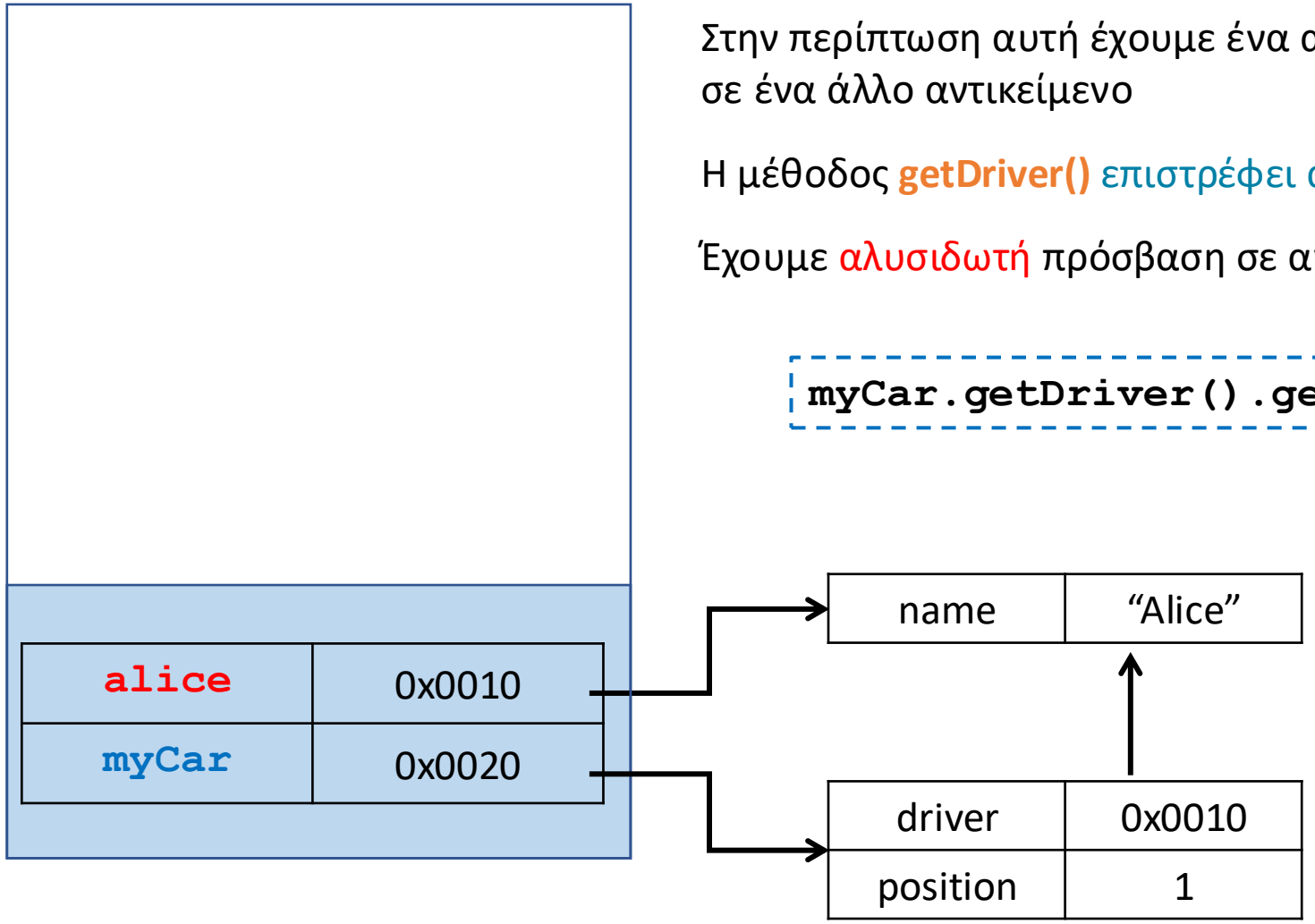
```
class Car
{
    private int position = 0;
    private Person driver;

    public Car(int position, Person driver){
        this.position = position;
        this.driver = driver;
    }

    public Person getDriver(){
        return driver;
    }
}
```

```
class MovingCarDriver1
{
    public static void main(String args[])
    {
        Person alice = new Person("Alice");
        Car myCar = new Car(1, alice);
        System.out.println(myCar.getDriver().getName());
    }
}
```

Dereferencing



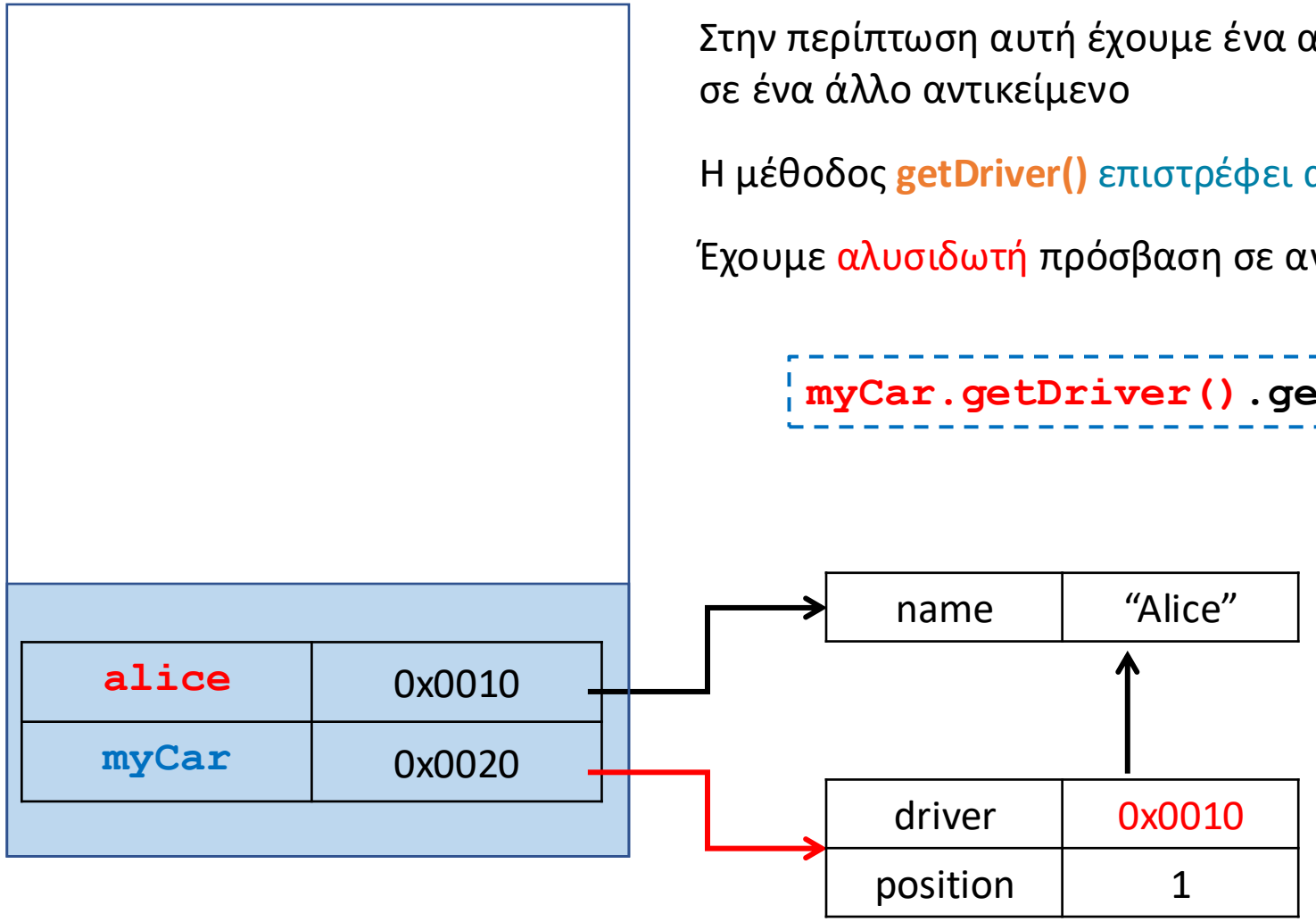
Στην περίπτωση αυτή έχουμε ένα αντικείμενο μέσα σε ένα άλλο αντικείμενο

Η μέθοδος **getDriver()** επιστρέφει αντικείμενο Person

Έχουμε αλυσιδωτή πρόσβαση σε αναφορές

```
myCar.getDriver().getName()
```

Dereferencing



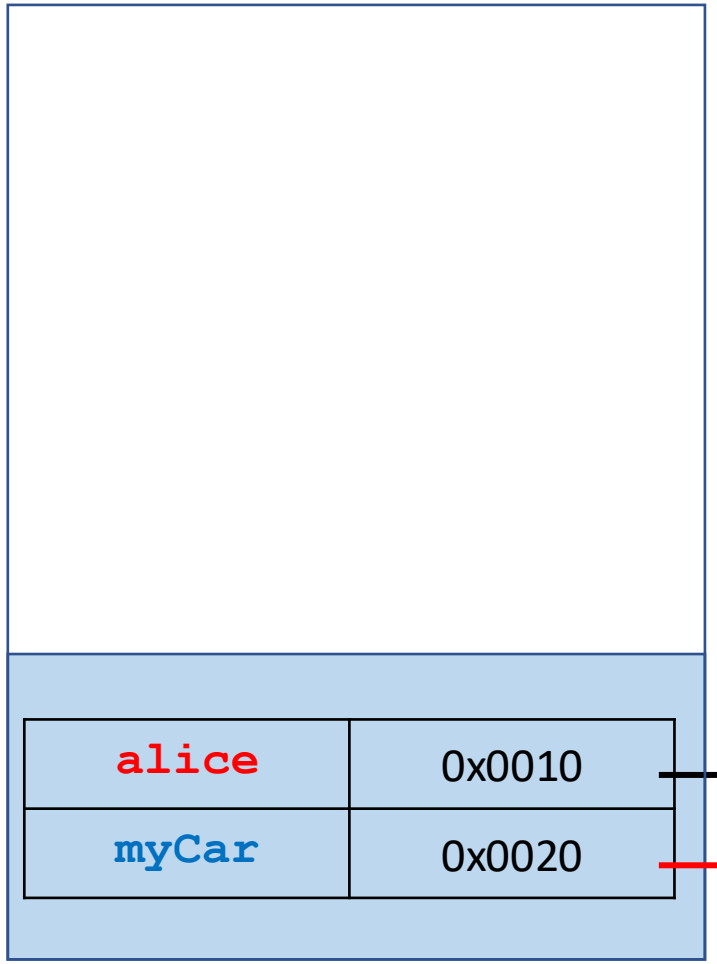
Στην περίπτωση αυτή έχουμε ένα αντικείμενο μέσα σε ένα άλλο αντικείμενο

Η μέθοδος **getDriver()** επιστρέφει αντικείμενο Person

Έχουμε αλυσιδωτή πρόσβαση σε αναφορές

```
myCar.getDriver().getName()
```

Dereferencing

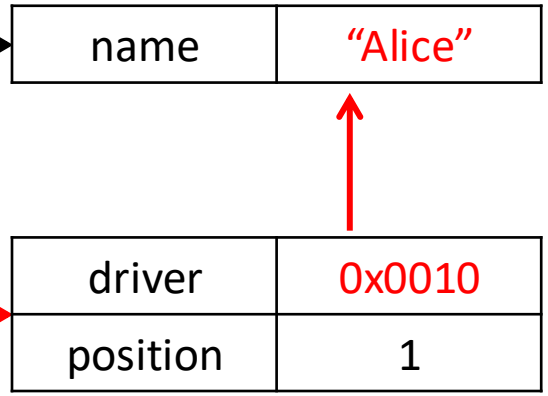


Στην περίπτωση αυτή έχουμε ένα αντικείμενο μέσα σε ένα άλλο αντικείμενο

Η μέθοδος **getDriver()** επιστρέφει αντικείμενο Person

Έχουμε αλυσιδωτή πρόσβαση σε αναφορές

```
myCar.getDriver().getName()
```



```
class Person
{
  private String name;

  public Person(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }
}
```

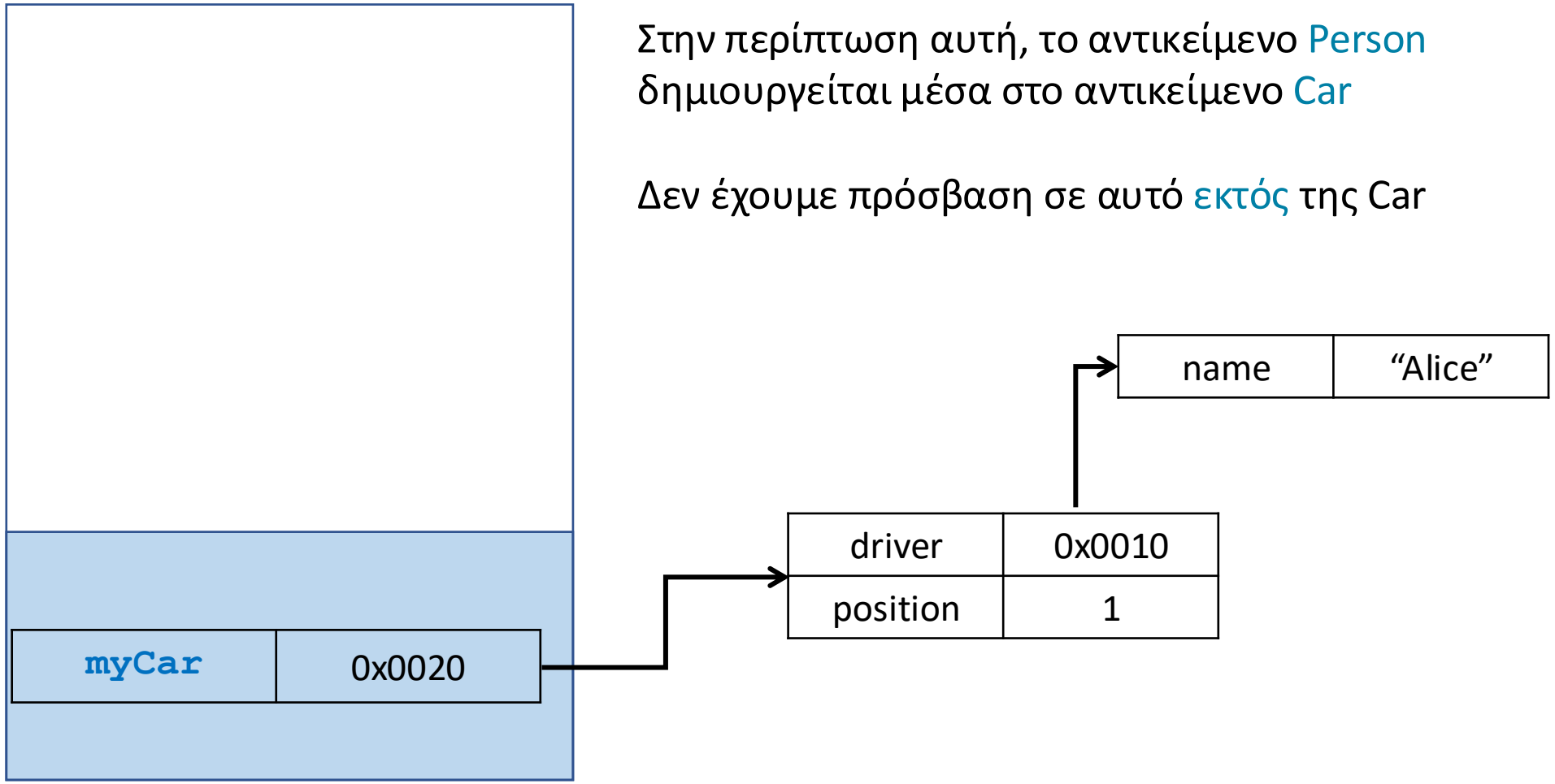
```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, String name){
    this.position = position;
    this.driver = new Person(name);
  }

  public String getDriverName(){
    return driver.getName();
  }
}
```

```
class MovingCarDriver2
{
  public static void main(String args[])
  {
    Car myCar = new Car(1, "Alice");
    System.out.println(myCar.getDriverName());
  }
}
```

Αντικείμενα μέσα σε αντικείμενα



Στην περίπτωση αυτή, το αντικείμενο Person δημιουργείται μέσα στο αντικείμενο Car

Δεν έχουμε πρόσβαση σε αυτό εκτός της Car

Σχέσεις μεταξύ κλάσεων

- Στο παράδειγμα μας έχουμε δύο διαφορετικές κλάσεις (Person, Driver) οι οποίες συσχετίζονται μεταξύ τους με διαφορετικούς τρόπους
- Μπορεί να υπάρχουν πολλές διαφορετικές σχέσεις μεταξύ κλάσεων
  - Στην περίπτωση μας, η μία κλάση ορίζεται χρησιμοποιώντας αντικείμενα της άλλης
- Αυτού του είδους τη σχέση την λέμε σχέση σύνθεσης
  - Μερικές φορές την ξεχωρίζουμε σε σχέση **σύνθεσης** (**composition**) και **συνάθροισης** (**aggregation**).

Σχέσεις κλάσεων

- Όταν έχουμε κλάσεις που έχουν αντικείμενα άλλων κλάσεων ένα θέμα που προκύπτει είναι πότε και πού θα γίνεται η δημιουργία των αντικειμένων και πότε η καταστροφή τους
- Τα αντικείμενα τύπου Person στο παράδειγμα MovingCarDriver2 δημιουργούνται **μέσα** στην κλάση Car. Αυτά επίσης καταστρέφονται μέσα στην Car, ή αν το αντικείμενο Car καταστραφεί
- Στο παράδειγμα MovingCarDriver1 αντικείμενα τύπου Person χρησιμοποιούνται στην Car αλλά δημιουργούνται **έξω** από αυτή την κλάση. Αυτά μπορεί να υπάρχουν αφού καταστραφεί η κλάση
- Συχνά οι σχέσεις του δεύτερου τύπου λέγονται σχέσεις **συνάθροισης**, ενώ σχέσεις του πρώτου τύπου **σύνθεσης**

Επιστροφή αντικειμένων

- Ένα αντικείμενο που δημιουργούμε μέσα σε μία μέθοδο μπορούμε να το διατηρήσουμε και μετά το τέλος της μεθόδου αν κρατήσουμε μια αναφορά σε αυτό
- Ένας τρόπος να γίνει αυτό είναι αν η μέθοδος επιστρέφει το αντικείμενο (δηλαδή την αναφορά σε αυτό) που δημιουργήσαμε

Η κλάση Date

```
class Date
{
    private int day = 1;
    private int month = 1;
    private int year = 2000;
    private String[] monthStrings =
                    {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                     "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

    public Date(int day, int month, int year){
       this.day = day;
       this.month = month;
       this.year = year;
    }

    public String toString(){
       return day + " " + monthNames[month-1] + " " + year;
    }
}
```

Θέλω η κλάση να μπορεί να μου επιστρέφει μια νέα ημερομηνία αλλά ένα χρόνο μετά. Πως μπορώ να το κάνω?

Η κλάση Date

```
class Date
{
    private int day = 1;
    private int month = 1;
    private int year = 2000;
    private String[] monthStrings =
                        {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

    public Date(int day, int month, int year){
        this.day = day; this.month = month; this.year = year;
    }

    public String toString(){
        return day + " " + monthNames[month-1] + " " + year;
    }

    public Date nextYear(){
        Date nextYearDate = new Date(day,month,year+1);
        return nextYearDate;
    }
}
```

Η μεθοδος **nextYear()** επιστρέφει ένα νέο αντικείμενο **Date** με την ημερομηνία ένα χρόνο μετά.

```
class DateExample
{
    public static void main(String args[]){
      Date today = new Date(19,2,2026);
      System.out.println(today);
      Date todayNextYear = today.nextYear();
      System.out.println(todayNextYear);
    }
}
```

Η κλάση Date

```
class Date
{
    private int day = 1;
    private int month = 1;
    private int year = 2000;
    private String[] monthStrings =
                        {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

    public Date(int day, int month, int year){
        this.day = day; this.month = month; this.year = year;
    }

    public String toString(){
        return day + " " + monthNames[month-1] + " " + year;
    }

    public Date nextYear(){
        return new Date(day,month,year+1);
    }
}
```

Τι γίνεται αν η ημερομηνία είναι 29/2?

Μπορούμε να επιστρέψουμε το αντικείμενο που δημιουργούμε κατευθείαν ως επιστρεφόμενη τιμή (παρομοίως και ως όρισμα σε μέθοδο)

Η κλάση Date

```
class Date
{
    private int day = 1;
    private int month = 1;
    private int year = 2000;
    private String[] monthStrings =
                        {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                        "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

    public Date(int day, int month, int year){
        this.day = day; this.month = month; this.year = year;
    }

    public String toString(){
        return day + " " + monthNames[month-1] + " " + year;
    }

    public Date nextYear(){
        if (day == 29 && month == 2){
            return null;
        }
        return new Date(day,month,year+1);
    }
}
```

Τι γίνεται αν η ημερομηνία είναι 29/2?

Η τιμή null: Μία κενή αναφορά. Η τιμή μπορεί να χρησιμοποιηθεί σαν μια default τιμή, ή σαν ένδειξη λάθους (στην περίπτωση αυτή ότι δεν μπορούμε να δημιουργήσουμε το αντικείμενο)

```
class DateExample
{
    public static void main(String args[]){
     Date today = new Date(19,2,2026);
     System.out.println(today);
     Date todayNextYear = today.nextYear();
     if( todayNextYear != null){
         System.out.println(todayNextYear);
     }
    }
}
```

Προσοχή: Η χρήση του null για έλεγχο λάθους σημαίνει ότι όποτε χρησιμοποιούμε την μέθοδο θα πρέπει να προσέχουμε αν η επιστρεφόμενη τιμή είναι null. Δεν είναι καλή λύση, και αργότερα θα μάθουμε για εξαιρέσεις για να χειριζόμαστε τέτοια προβλήματα.

Πίνακες από αντικείμενα

- Όπως ορίζουμε πίνακες από πρωταρχικούς τύπους μπορούμε να ορίσουμε και πίνακες από αντικείμενα
  - `Person[] array = new Person[3];`
  - Ορίζει ένα πίνακα με τρία αντικείμενα τύπου Person
  - Ουσιαστικά ένα πίνακα με αναφορές

- Όταν ορίζουμε ένα πίνακα από αντικείμενα πρέπει να είμαστε προσεκτικοί να δεσμεύουμε σωστά τη μνήμη

Παράδειγμα

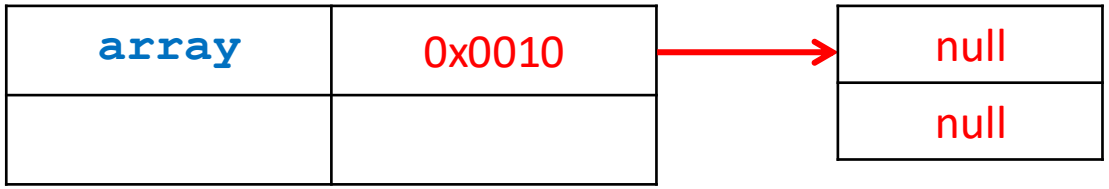
```
Person[] array;
```

array	null

- Η εντολή αυτή θα δημιουργήσει μια μεταβλητή με το όνομα **array** η οποία κάποια στιγμή θα δείχνει σε ένα πίνακα με **Person**. Για την ώρα είναι **null**

Παράδειγμα

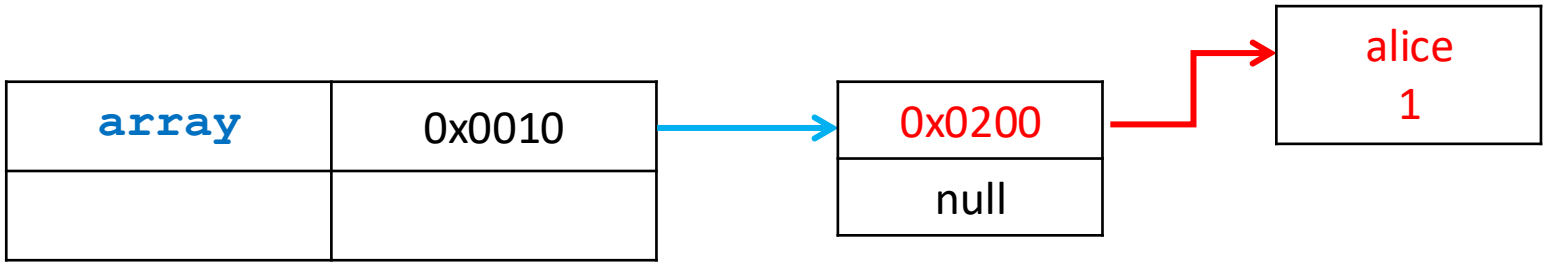
```
Person[] array;
array = new Person[2];
```



- Η εντολή **new** θα δεσμεύσει δύο θέσεις μνήμης στο heap για να κρατήσουν δύο αναφορές τύπου **Person**. Εφόσον δεν έχουμε δημιουργήσει τις μεταβλητές ακόμη, αυτές θα είναι **null**

Παράδειγμα

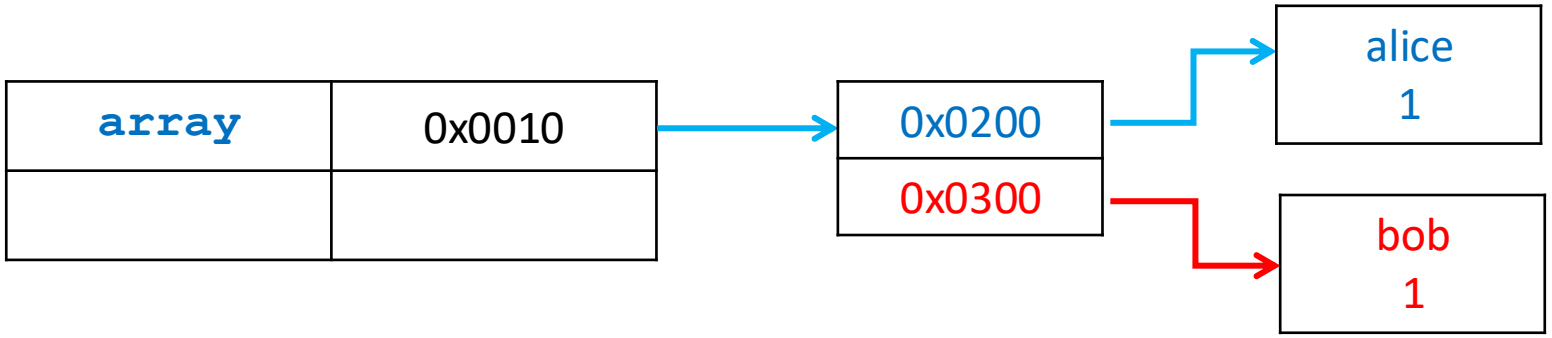
```
Person[] array;
array = new Person[2];
array[0] = new Person("alice", 1);
```



- Η νέα εντολή new θα δεσμεύσει χώρο για ένα **Person**. Δημιουργείται το αντικείμενο και η αναφορά αποθηκεύεται στην πρώτη θέση του πίνακα **array**

Παράδειγμα

```
Person[] array;
array = new Person[2];
array[0] = new Person("alice", 1);
array[1] = new Person("bob", 1);
```



- Η νέα εντολή **new** θα δεσμεύσει χώρο για άλλο ένα **Person**. Δημιουργείται το αντικείμενο και η αναφορά αποθηκεύεται στην δεύτερη θέση του πίνακα **array**.

Πίνακες από πίνακες

- Οι δισδιάστατοι πίνακες είναι ουσιαστικά πίνακες από αντικείμενα, όπου τα αντικείμενα είναι πάλι πίνακες
- Π.χ., έτσι δεσμεύουμε πίνακα ακεραίων 5×5

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```

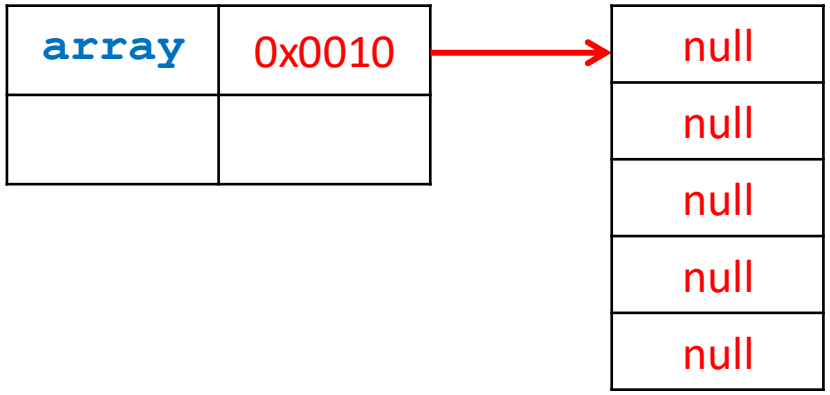
Παράδειγμα

```
int[][] array;
```

array	null

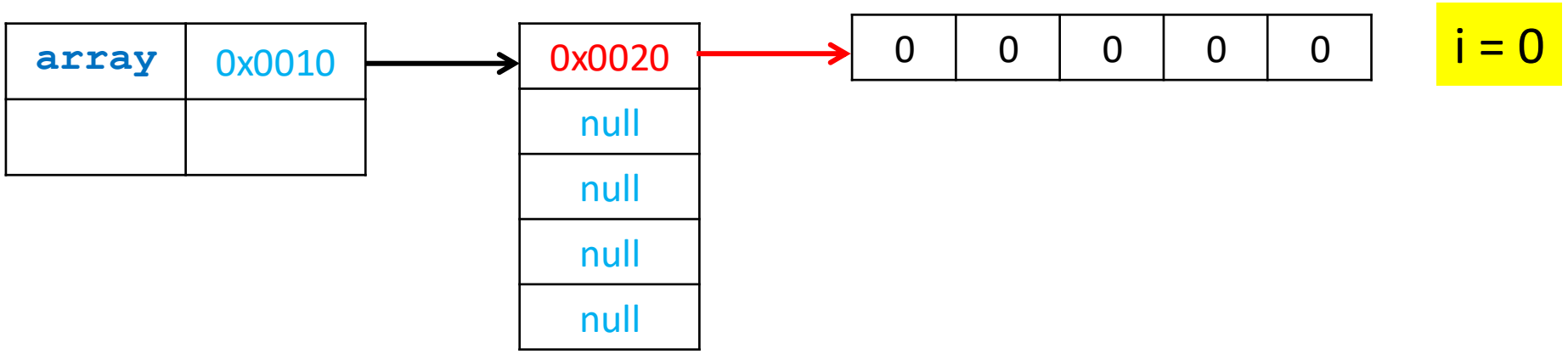
Παράδειγμα

```
int[][] array;
array = new int[5][];
```



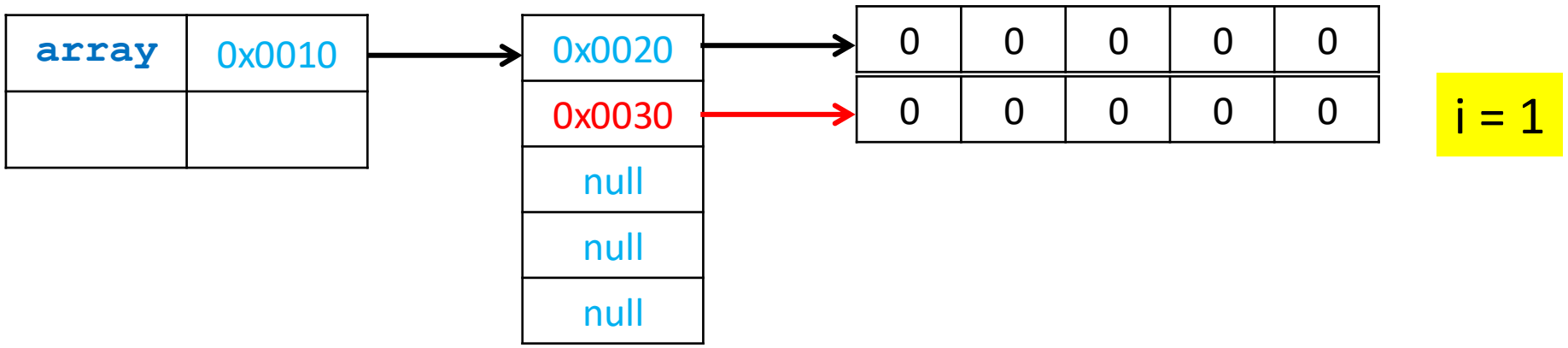
Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```



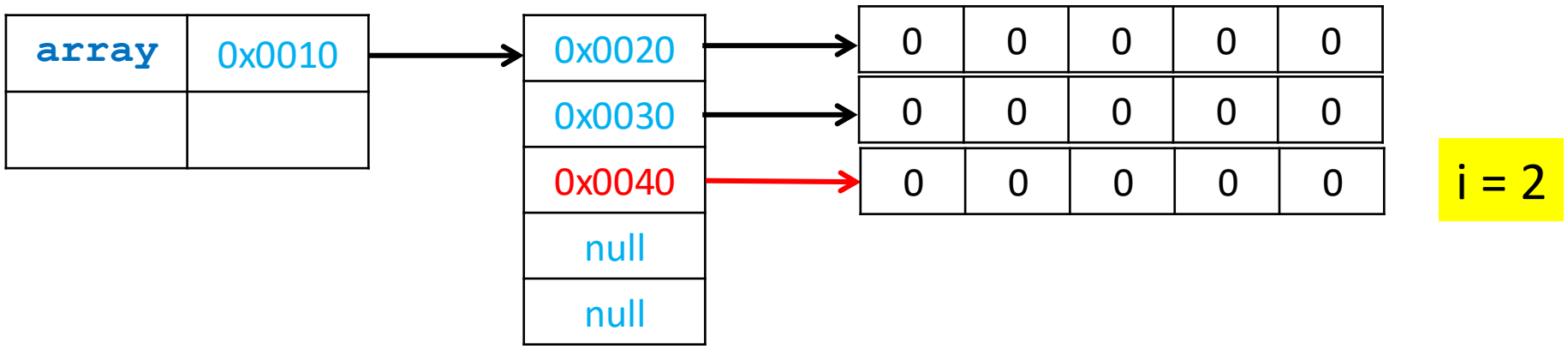
Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```



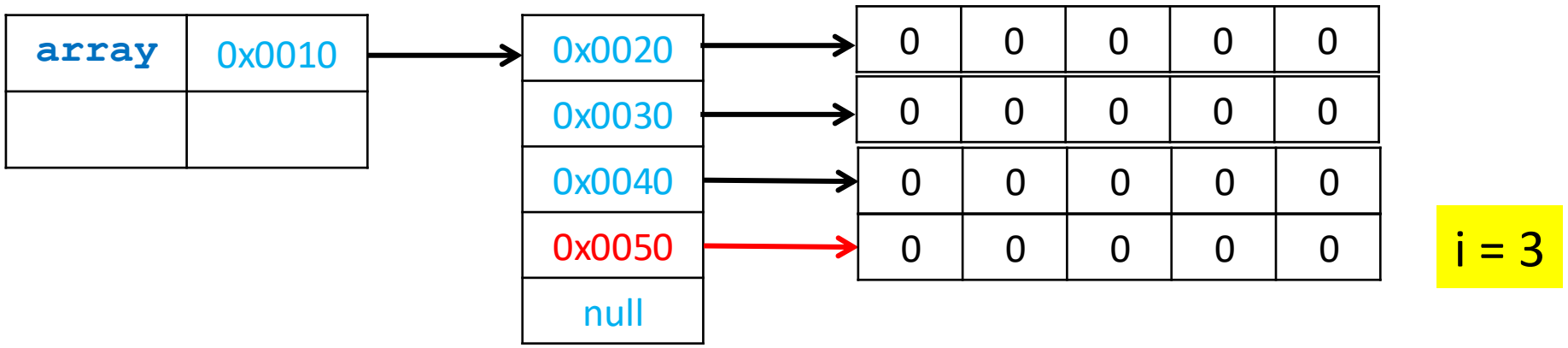
Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```



Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```



Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[5];
}
```



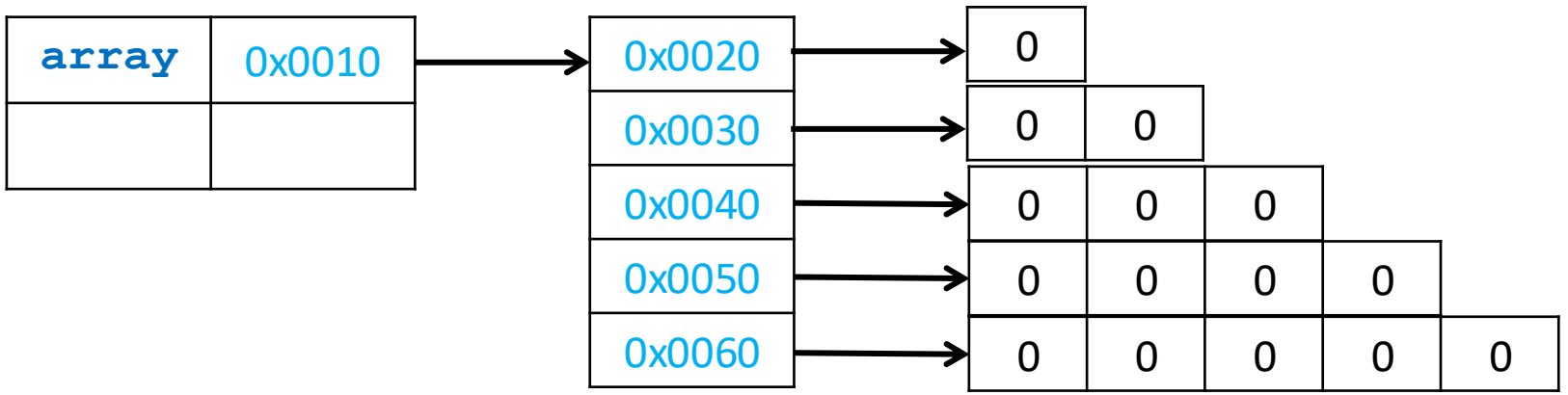
Πίνακες από πίνακες

- Μπορεί ο δισδιάστατος μας πίνακας να είναι ασύμμετρος.
- Π.χ., έτσι ορίζουμε ένα διαγώνιο πίνακα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[i+1];
}
```

Παράδειγμα

```
int[][] array;
array = new int[5][];
for (int i=0; i<5; i++){
      array[i] = new int[i+1];
}
```



```
public class Person
{
    private String name;
    private int number;

    public Person(String initName, int initNumber){
        name = initName;
        number = initNumber;
    }

    public void set(String newName, int newNumber){
        name = newName;
        number = newNumber;
    }

    public String toString( ){
        return (name + " " + number);
    }

    public Person copier() {
        Person newPerson = new Person(this.name,this.number);
        return newPerson;
    }

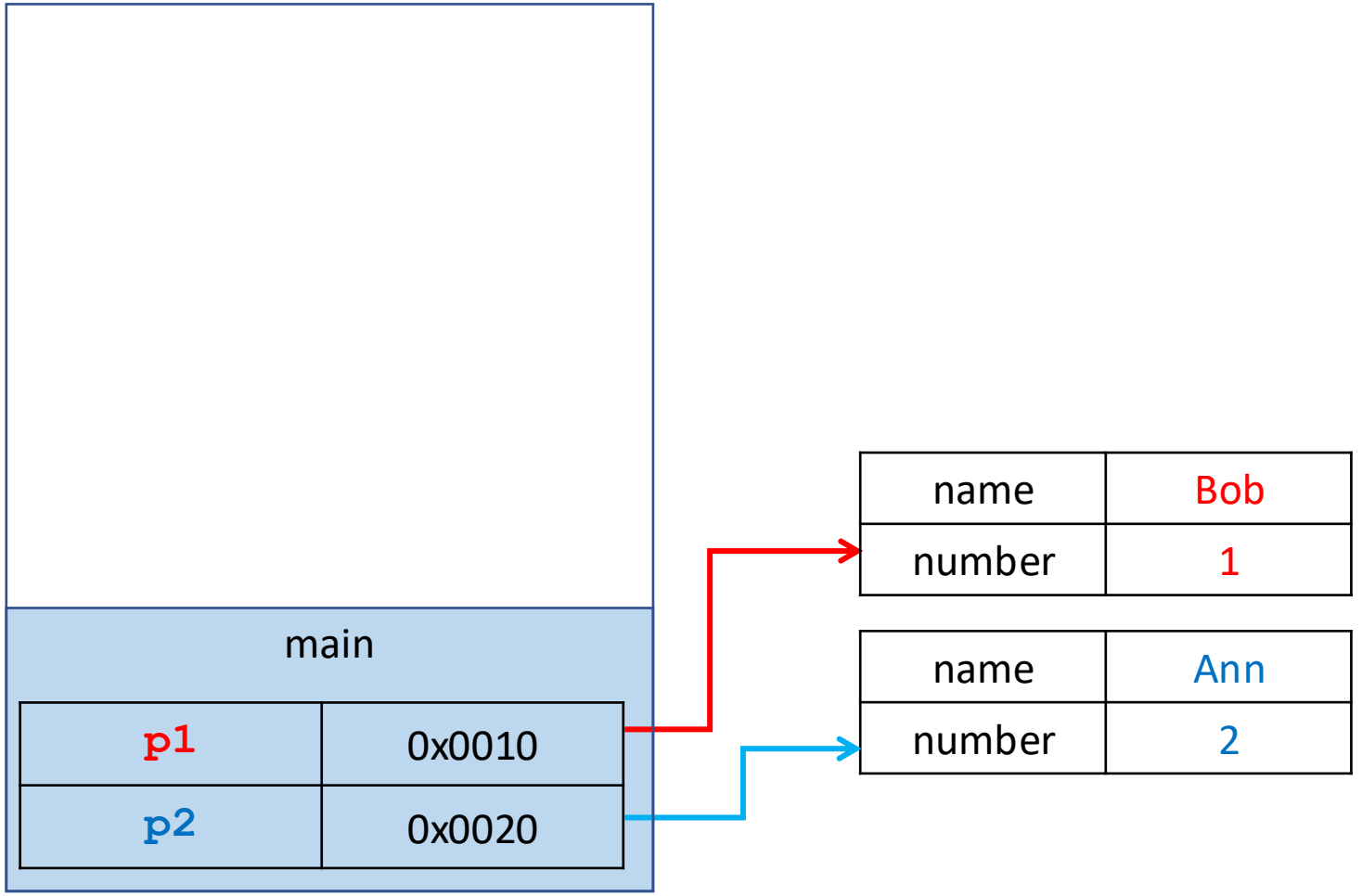
}
```

Παράδειγμα

```
public class ClassParameterDemo
{
    public static void main(String[] args)
    {
        Person p1 = new Person("Bob", 1);
        Person p2 = new Person("Ann", 2);
        p1 = p2.copier();
        System.out.println(p1);
     }
}
```

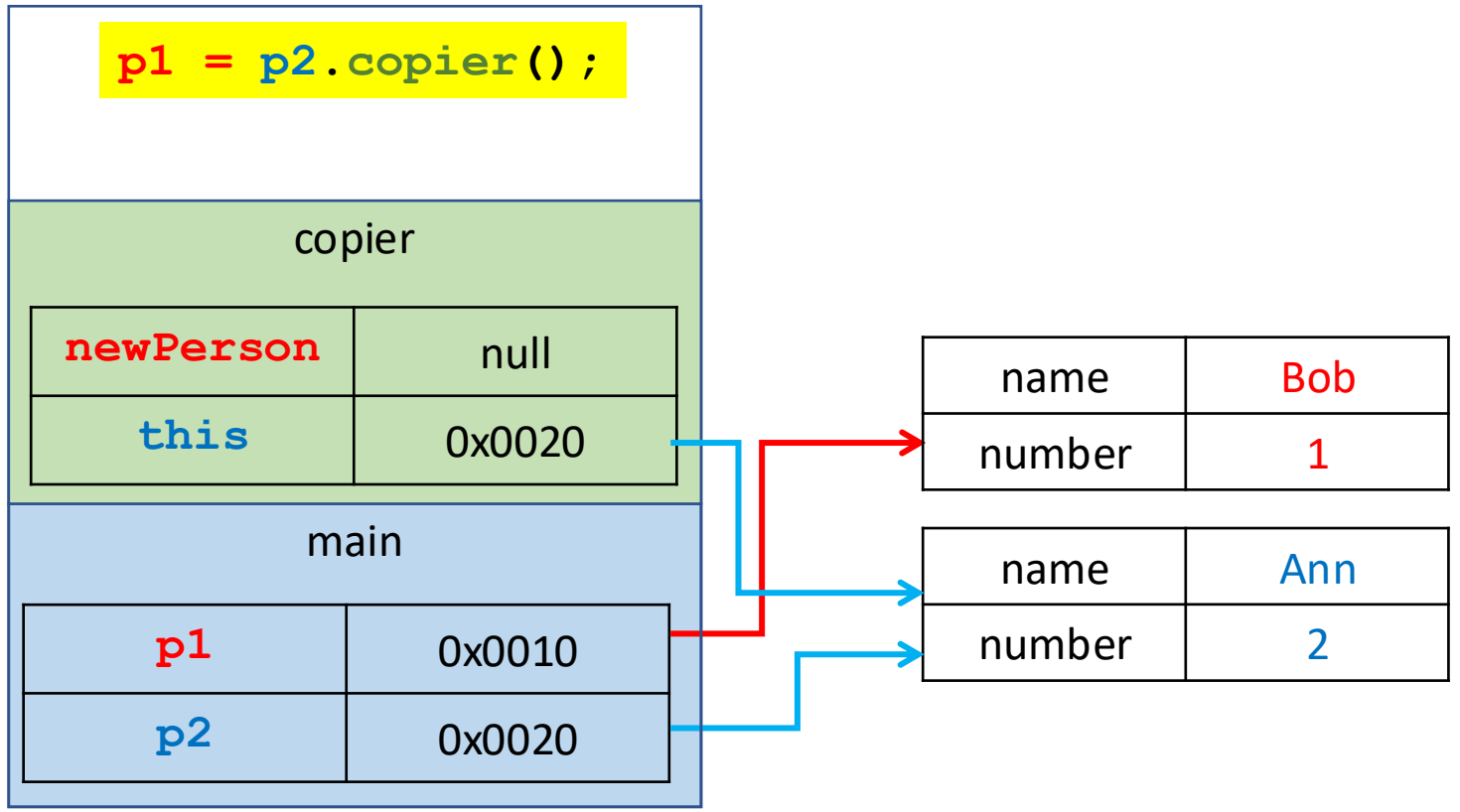
Τι θα τυπώσει?

Εξέλιξη του προγράμματος



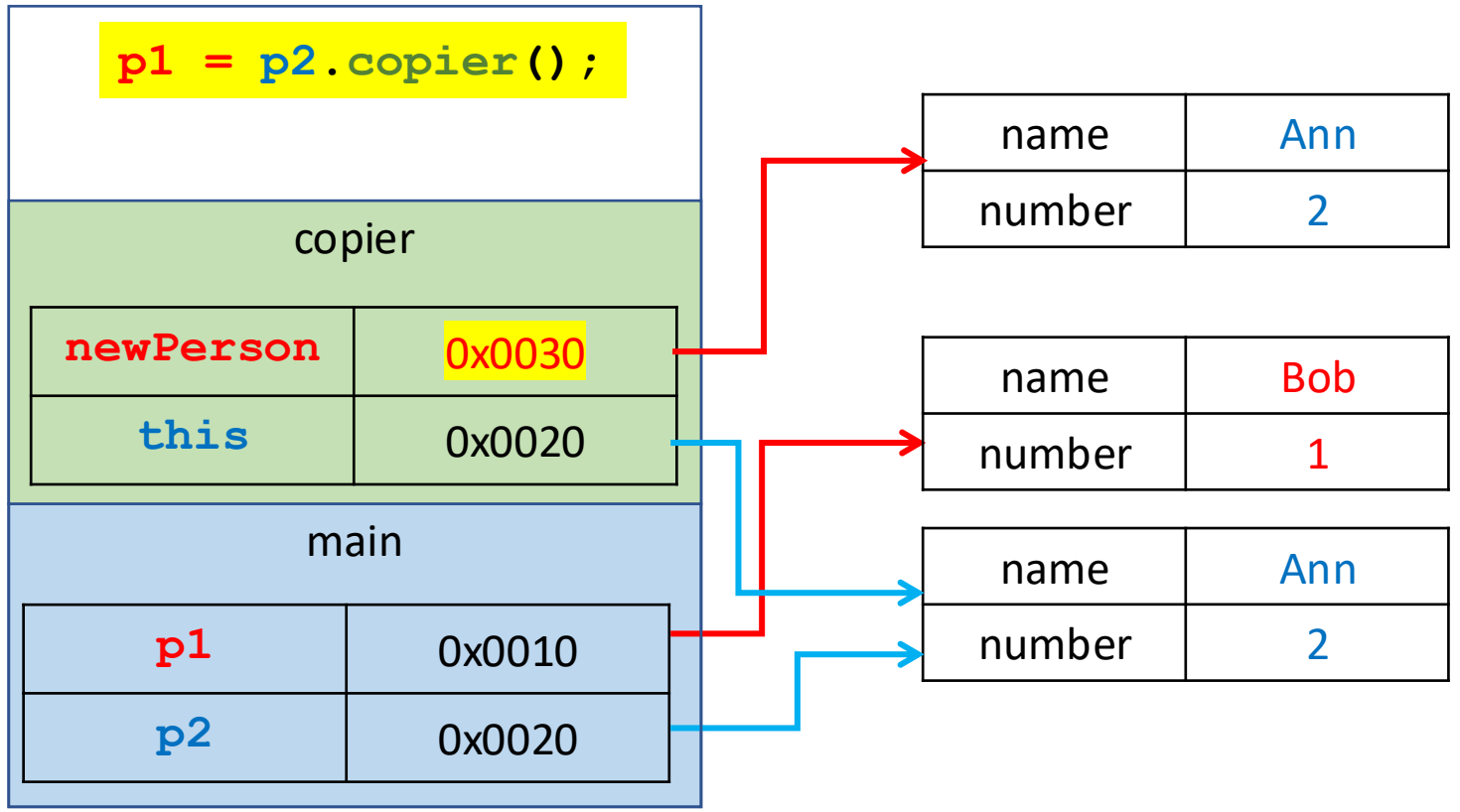
Εξέλιξη του προγράμματος

```
public Person copier() {
    Person newPerson = new Person(this.name,this.number);
    return newPerson;
}
```



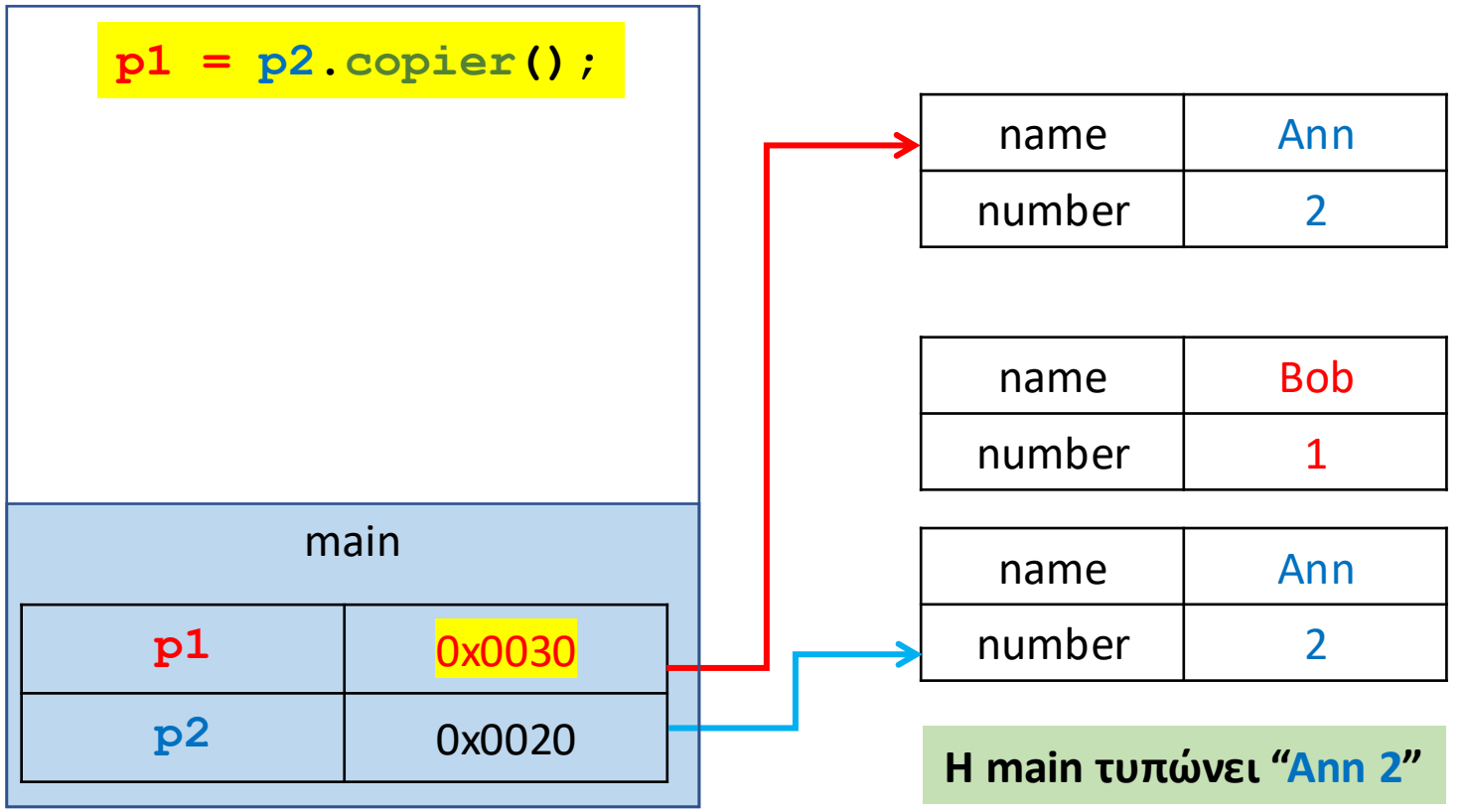
Εξέλιξη του προγράμματος

```
public Person copier() {
    Person newPerson = new Person(this.name,this.number);
    return newPerson;
}
```



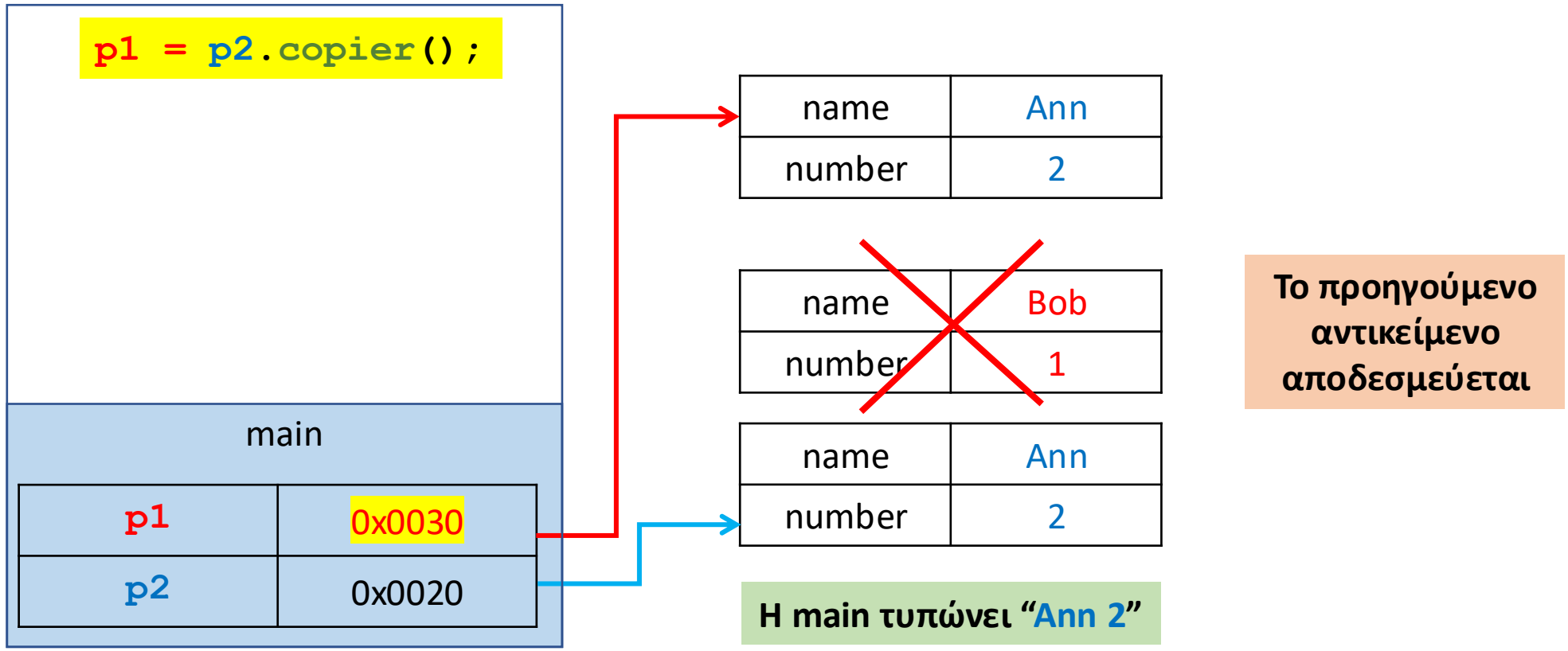
Εξέλιξη του προγράμματος

```
public Person copier() {
    Person newPerson = new Person(this.name,this.number);
    return newPerson;
}
```



Εξέλιξη του προγράμματος

```
public Person copier() {
    Person newPerson = new Person(this.name,this.number);
    return newPerson;
}
```



Δημιουργία αντιγράφων

- Η μέθοδος **copier** όπως την ορίσαμε πριν δημιουργεί ένα **καινούριο αντικείμενο** που είναι **αντίγραφο** αυτού που έκανε την κλήση
- Στην περίπτωση μας το αντικείμενο έχει μόνο πεδία που είναι **πρωταρχικού τύπου** ή **μη μεταλλάξιμα αντικείμενα (immutable)**. Γενικά ένα αντικείμενο μπορεί να έχει ως πεδία άλλα **αντικείμενα** (δηλαδή αναφορές)
- Στην περίπτωση αυτή η **δημιουργία αντιγράφου** θα πρέπει να γίνεται με πολύ **προσοχή**!

```
class Car
{
    private int dim;
    private int[] position;

    public Car(int d){
       dim = d; position = new int[d];
    }

    public void move(){
       for (int i=0; i < dim; i++){position[i]++;}
    }

    public Car copy(){
       Car newCar = new Car(this.dim);
       newCar.position = this.position;
       return newCar;
    }

    public String toString(){
       String output = "";
       for (int i=0; i < dim; i++){output = output + position[i] + " ";}
       return output;
    }

     public static void main(String args[]){
          Car car1 = new Car(2);
          car1.move();
          Car car2 = car1.copy();
          car2.move();
          System.out.println(car1);
    }
}
```

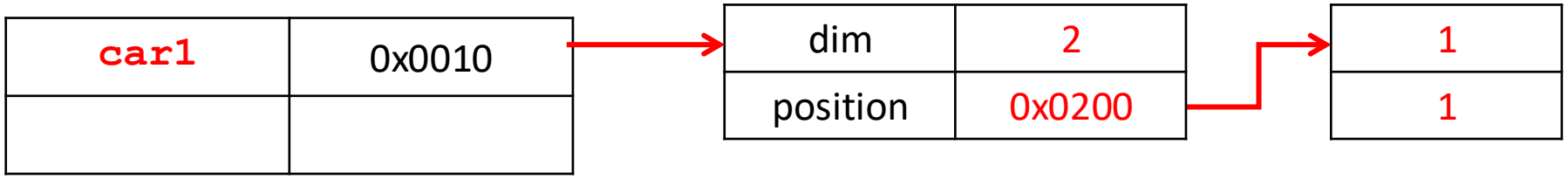
Το Car κινείται σε 1 ή 2 διαστάσεις
Χρειαζόμαστε ένα πίνακα για την θέση του

Η copy δημιουργεί και επιστρέφει ένα νέο Car

Τι θα τυπώσει η main? 2 2

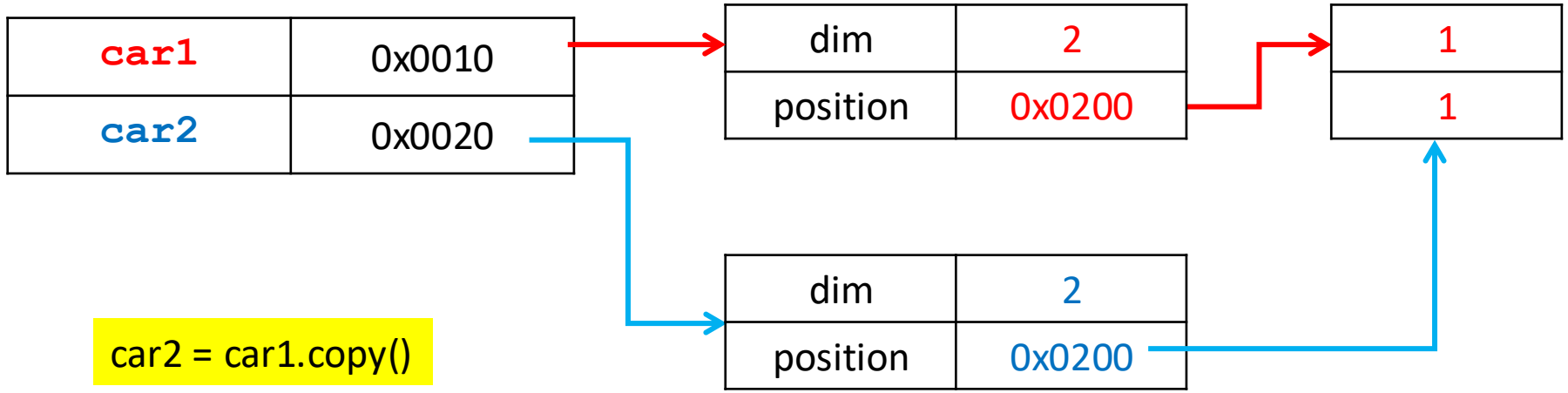
Ρηχά Αντίγραφα (Shallow Copy)

- Η **copy** όπως την έχουμε ορίσει δημιουργεί ένα **ρηχό αντίγραφο (shallow copy)** του αντικειμένου
  - Αντιγράφει τις αναφορές στα αντικείμενα και όχι τα περιεχόμενα των αντικειμένων



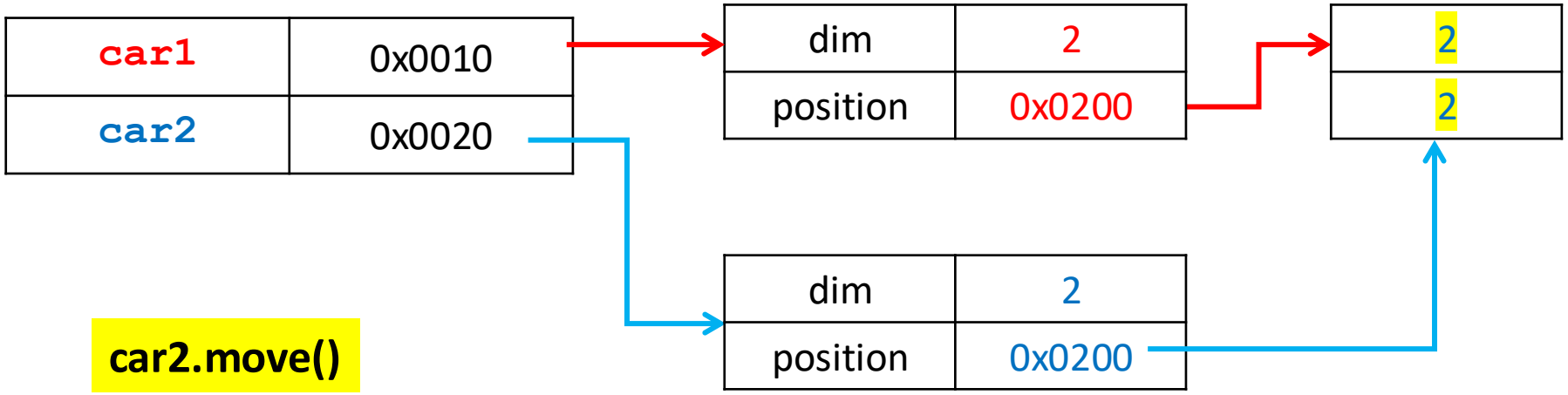
Ρηχά Αντίγραφα (Shallow Copy)

- Η **copy** όπως την έχουμε ορίσει δημιουργεί ένα **ρηχό αντίγραφο (shallow copy)** του αντικειμένου
  - Αντιγράφει τις αναφορές στα αντικείμενα και όχι τα περιεχόμενα των αντικειμένων



Ρηχά Αντίγραφα (Shallow Copy)

- Η **copy** όπως την έχουμε ορίσει δημιουργεί ένα **ρηχό αντίγραφο (shallow copy)** του αντικειμένου
  - Αντιγράφει τις αναφορές στα αντικείμενα και όχι τα περιεχόμενα των αντικειμένων

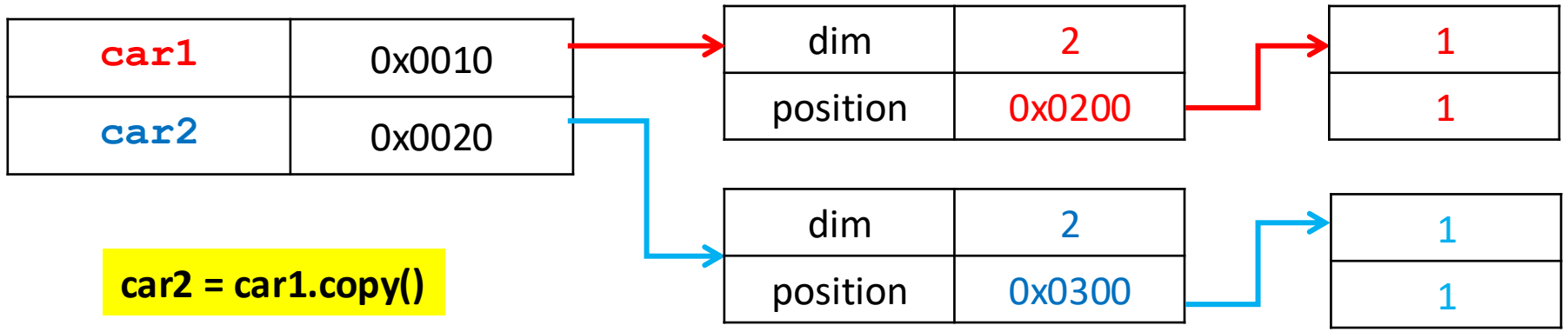


Μετακινείται και το car1 αλλά αυτό δεν είναι επιθυμητό

Βαθύ Αντίγραφο (Deep Copy)

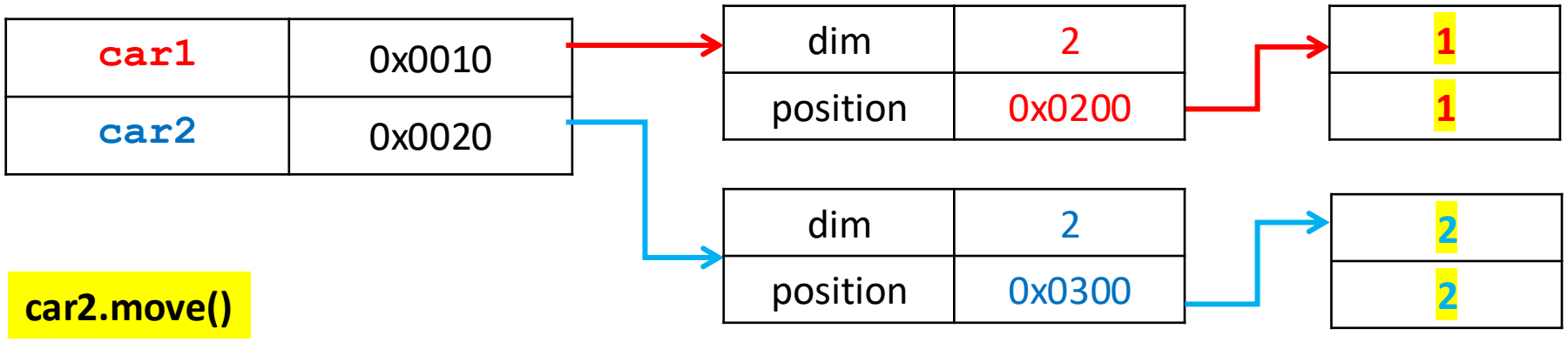
- Τις περισσότερες φορές θέλουμε να κάνουμε ένα **βαθύ αντίγραφο** του αντικειμένου, όπου για κάθε αντικείμενο μέσα στο αντίγραφο **δεσμεύουμε νέα μνήμη**

```
public Car copy(){
    Car newCar = new Car(this.dim);
    for (int i=0; i<dim; i++){
        newCar.position[i] = this.position[i];
    }
    return newCar;
}
```



Βαθύ Αντίγραφο (Deep Copy)

- Το **βαθύ αντίγραφο** του car1 είναι πλέον ένα ανεξάρτητο αντικείμενο



Η μετακίνηση του car2 δεν επηρεάζει το car1

Copy Constructor

- Ένας Constructor που παίρνει σαν όρισμα ένα αντικείμενο του ίδιου τύπου και δημιουργεί ένα αντίγραφο
  - `public Car(Car other)`

- Ο **copy constructor** έχει δύο λειτουργίες:
  - Δεσμεύει τη μνήμη για το αντικείμενο
  - Αντιγράφει τις τιμές του αντικειμένου-ορίσματος
- Πάντα πρέπει να δημιουργούμε ένα **βαθύ αντίγραφο** του αντικειμένου

Copy Constructor για την Car

```
public Car(Car other)
{
    this.dim = other.dim;
    position = new int[this.dim];
    for (int i = 0; i < this.dim; i++){
        this.position[i] = other.position[i];
    }
}
```

Κλήση:

```
Car car1 = new Car(2);
Car car2 = new Car(car1);
```

Δημιουργεί **βαθύ αντίγραφο.** Δεσμεύουμε καινούριο πίνακα και αντιγράφουμε μία-μία τις τιμές

Φωλιασμένος Copy Constructor

- Αν μια κλάση έχει πεδία **αντικείμενα** από μία **άλλη κλάση**, τότε όταν καλούμε τον copy constructor θα πρέπει να έχουμε ορίσει **copy constructor** και για τις κλάσεις των αντικειμένων-πεδίων.

Παράδειγμα

```
public class CarDriver
{
    private int position;
    private Person driver;

    public CarDriver(CarDriver other){
        this.position = other.position;
        driver = new Person(other.driver);
    }
}
```

Καλεί την copy constructor της Person

```
public class Person
{
    private String name;
    private int number;

    public Person(String initName, int initNumber){
        name = initName;number = initNumber;
    }

    public Person(Person other){
        this.name = other.name;
        this.number = other.number;
    }

    public void set(String newName, int newNumber){
        name = newName;
        number = newNumber;
    }

    public String toString( ){
        return (name + " " + number);
    }

    public boolean equals(Person other){
       return (this.name.equals(other.name) && this.number == other.number;
    }
}
```

Φωλιασμένη equals

```
public class CarDriver
{
      private int position;
      private Person driver;

      public CarDriver(CarDriver other){
          this.position = other.position;
          driver = new Person(other.driver);
      }

        public boolean equals(CarDriver other){
          return this.driver.equals(other.driver) && this.position == other.position;
        }
}
```

Καλεί την equals της Person

Φωλιασμένη toString()

```
public class CarDriver
{
      private int position;
      private Person driver;

      public CarDriver(CarDriver other){
         this.position = other.position;
         driver = new Person(other.driver);
      }

       public boolean equals(CarDriver other){
          return this.driver.equals(other.driver)
             && this.position == other.position;
       }

       public String toString(){
          return driver + " " + position;
       }
}
```

Καλεί την toString της Person

Παραδείγματα

- Τι γίνεται αν έχουμε ένα constructor που παίρνει όρισμα ένα πίνακα?
  - `public Car(int[] position)`
  - Αν ο πίνακας αλλάξει μέσα στην main θα αλλάξει και στο αντικείμενο.

- Τι γίνεται αν κάνουμε τον πίνακα null στην main?

```
class CarArray
{
    private int[] position;
    private int dim;

    public CarArray(int[] array){
        dim = array.length;
        position = array;
    }

    public void move(){
        for (int i=0; i < dim; i++){
            position[i]++;
        }
    }

    public String toString(){
        String output = "";
        for (int i=0; i < dim; i++){
            output = output + position[i] + " ";
        }
        return output;
    }

    public static void main(String args[]){
        int[] pos = {1,2};
        CarArray myCar = new CarArray(pos);
        myCar.move();
        pos[0]++;
        pos = null;
    }
}
```

```
System.out.println(pos[0]+ " " + pos[1]);
System.out.println(myCar);
System.out.println(myCar);
```

Τι θα τυπώσει?

```
2 3
3 3
3 3
```

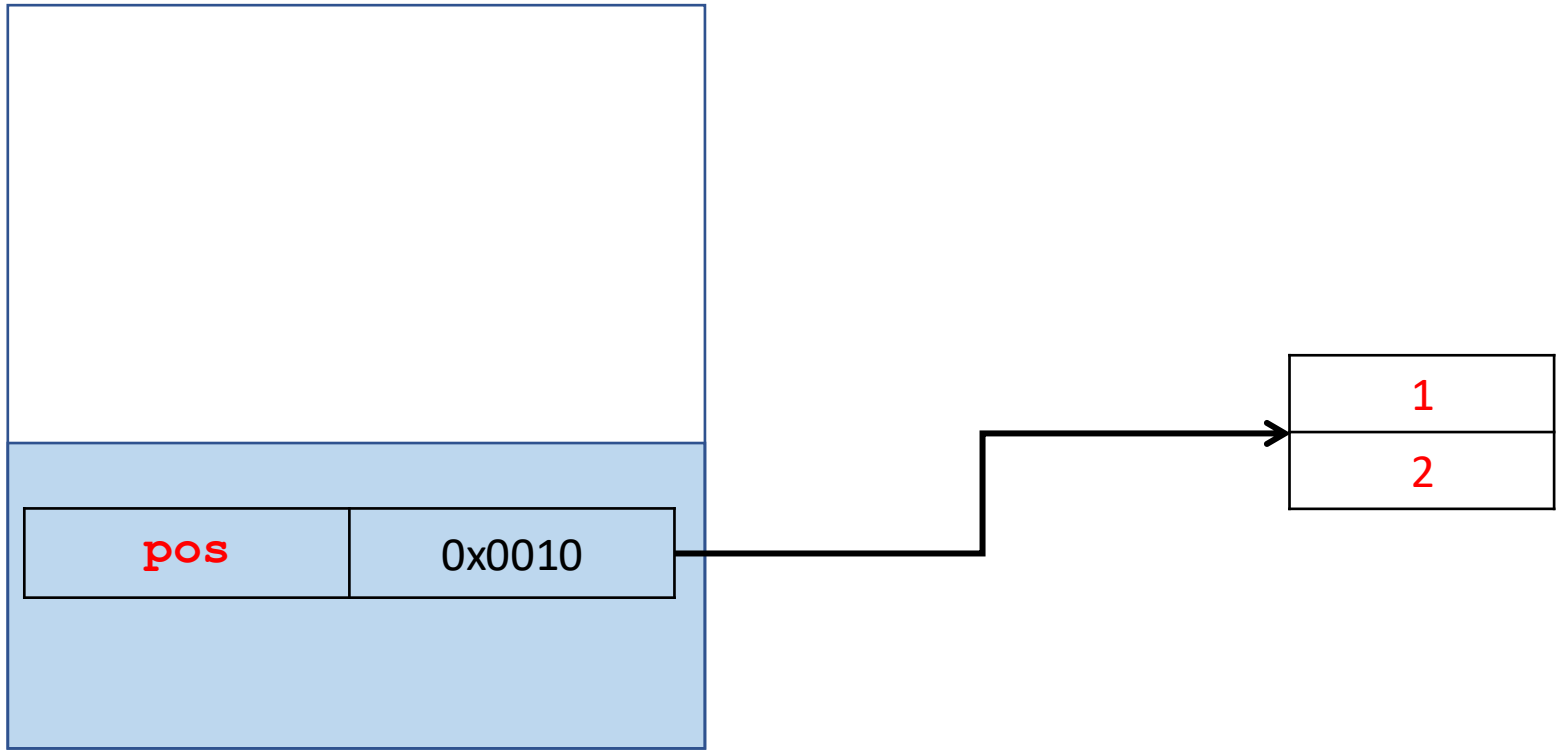
Η αλλαγή στα περιεχόμενα του πίνακα στο αντικείμενο myCar αλλάζει και τα περιεχόμενα του pos

Η αλλαγή στα περιεχόμενα του pos αλλάζει και τα περιεχόμενα του πίνακα στο αντικείμενο myCar

Η αλλαγή στην τιμή του pos δεν αλλάζει τα περιεχόμενα το πεδίο στο αντικείμενο myCar

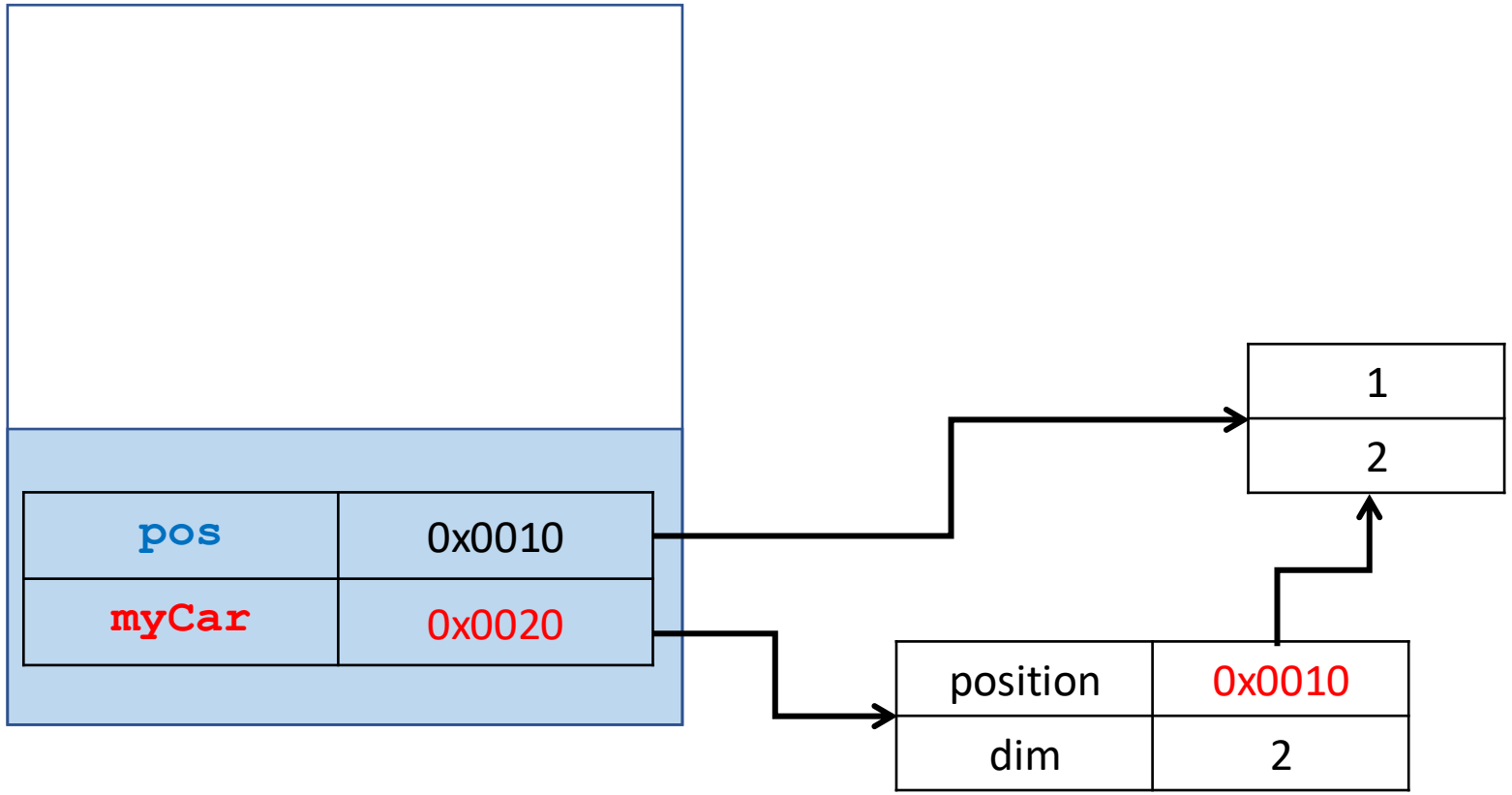
Εκτέλεση

```
int[] pos = {1,2}
```



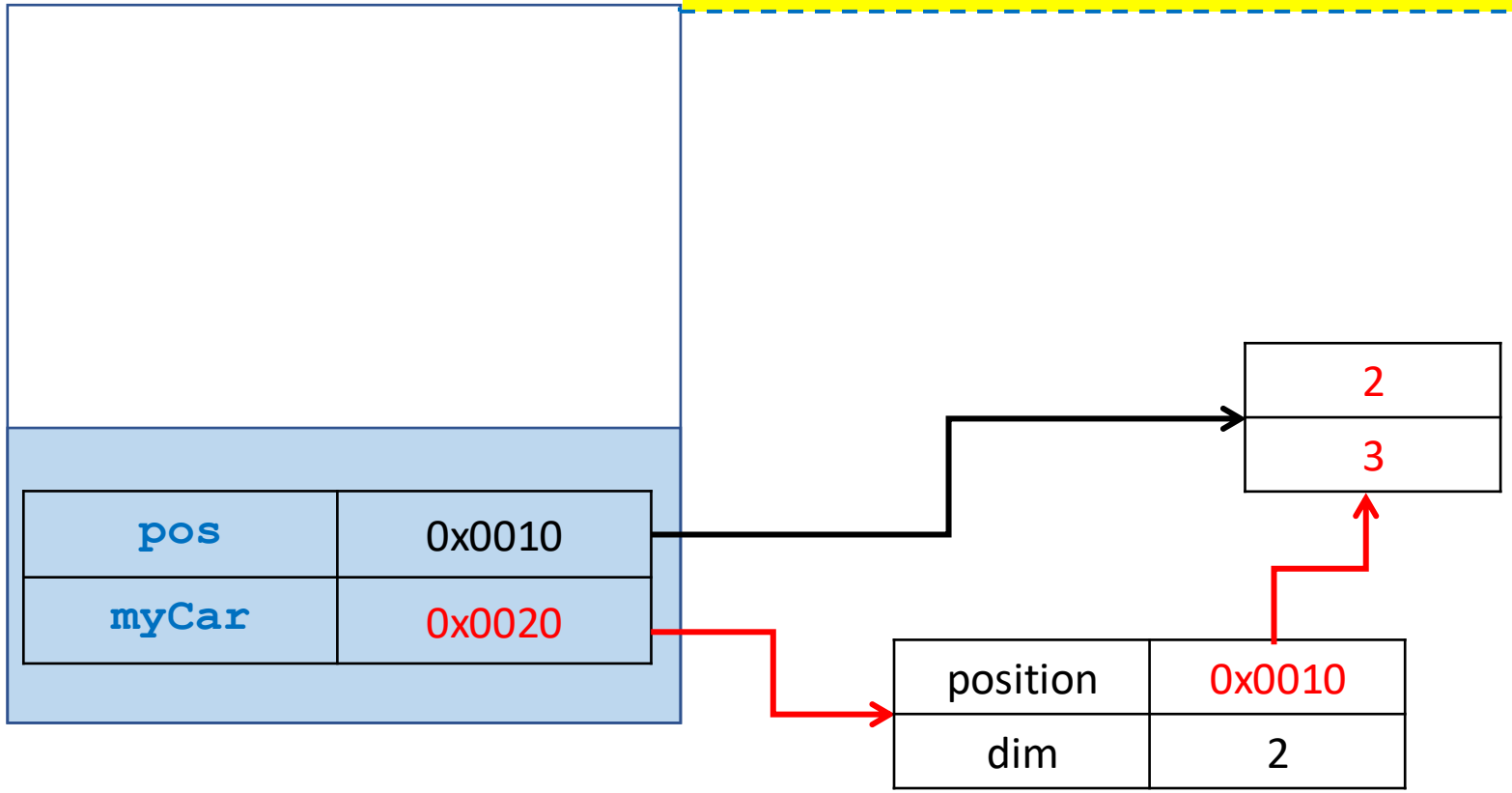
Εκτέλεση

```
int[] pos = {1,2}
CarArray myCar = new CarArray(pos);
```



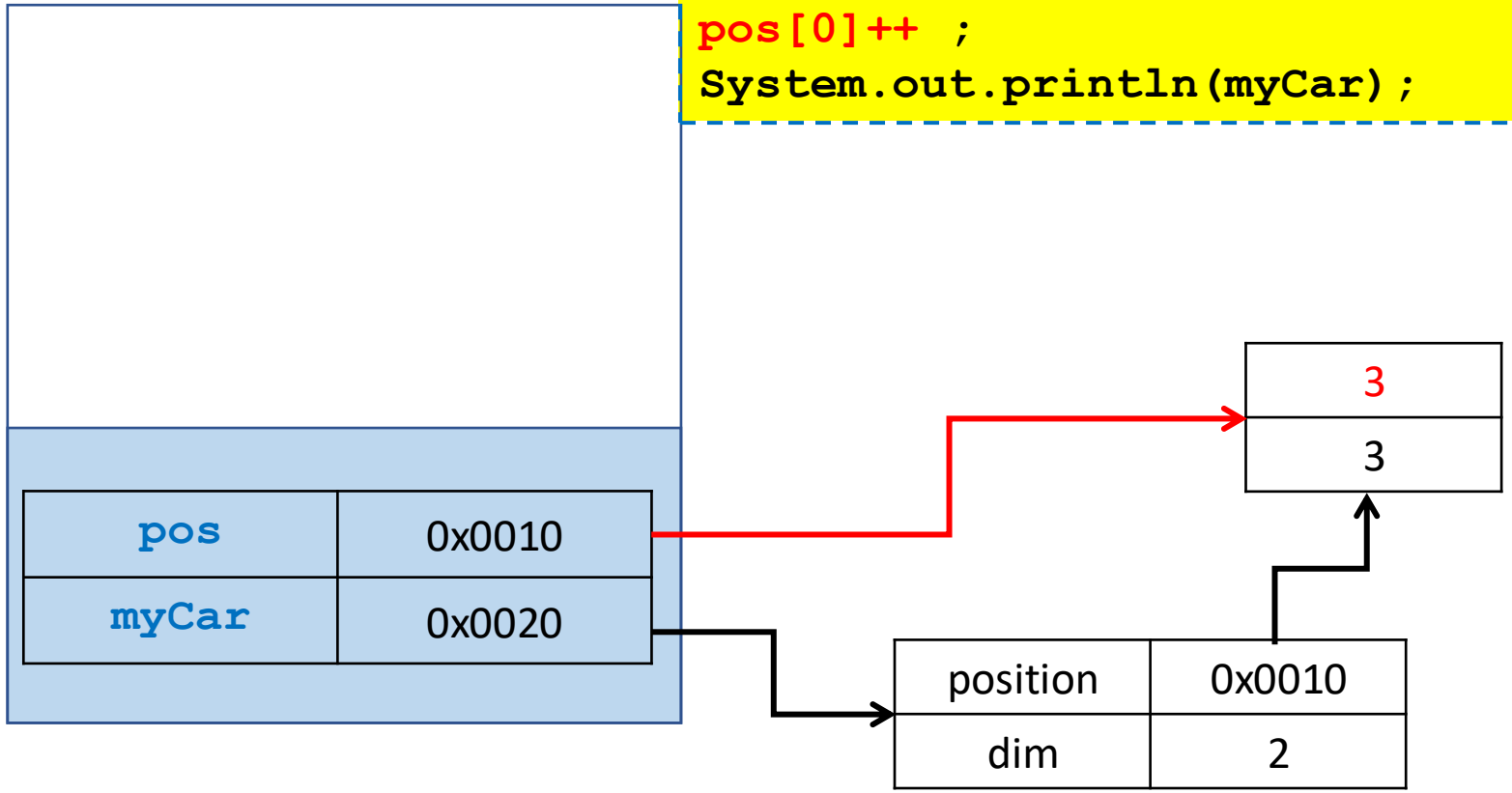
Εκτέλεση

```
int[] pos = {1,2}
CarArray myCar = new CarArray(pos);
myCar.move();
System.out.println(pos[0]+" "+pos[1]);
```



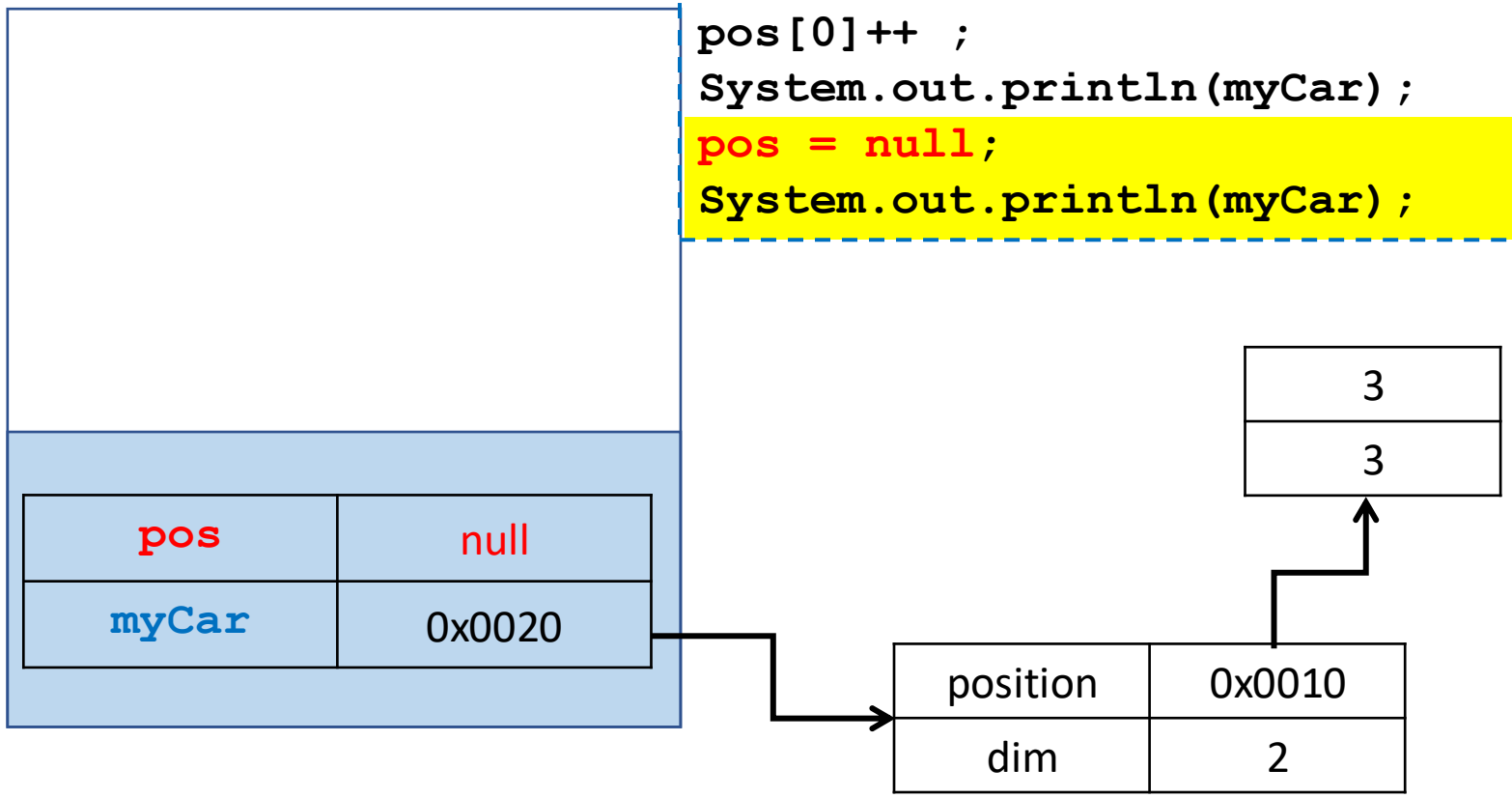
Εκτέλεση

```
int[] pos = {1,2}
CarArray myCar = new CarArray(pos);
myCar.move();
System.out.println(pos[0]+" "+pos[1]);
pos[0]++ ;
System.out.println(myCar);
```



Εκτέλεση

```
int[] pos = {1,2}
CarArray myCar = new CarArray(pos);
myCar.move();
System.out.println(pos[0]+" "+pos[1]);
pos[0]++ ;
System.out.println(myCar);
pos = null;
System.out.println(myCar);
```



```
class Person
{
  private String name;

  public Person(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }

  public void setName(String name)
  {
    this.name = name;
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

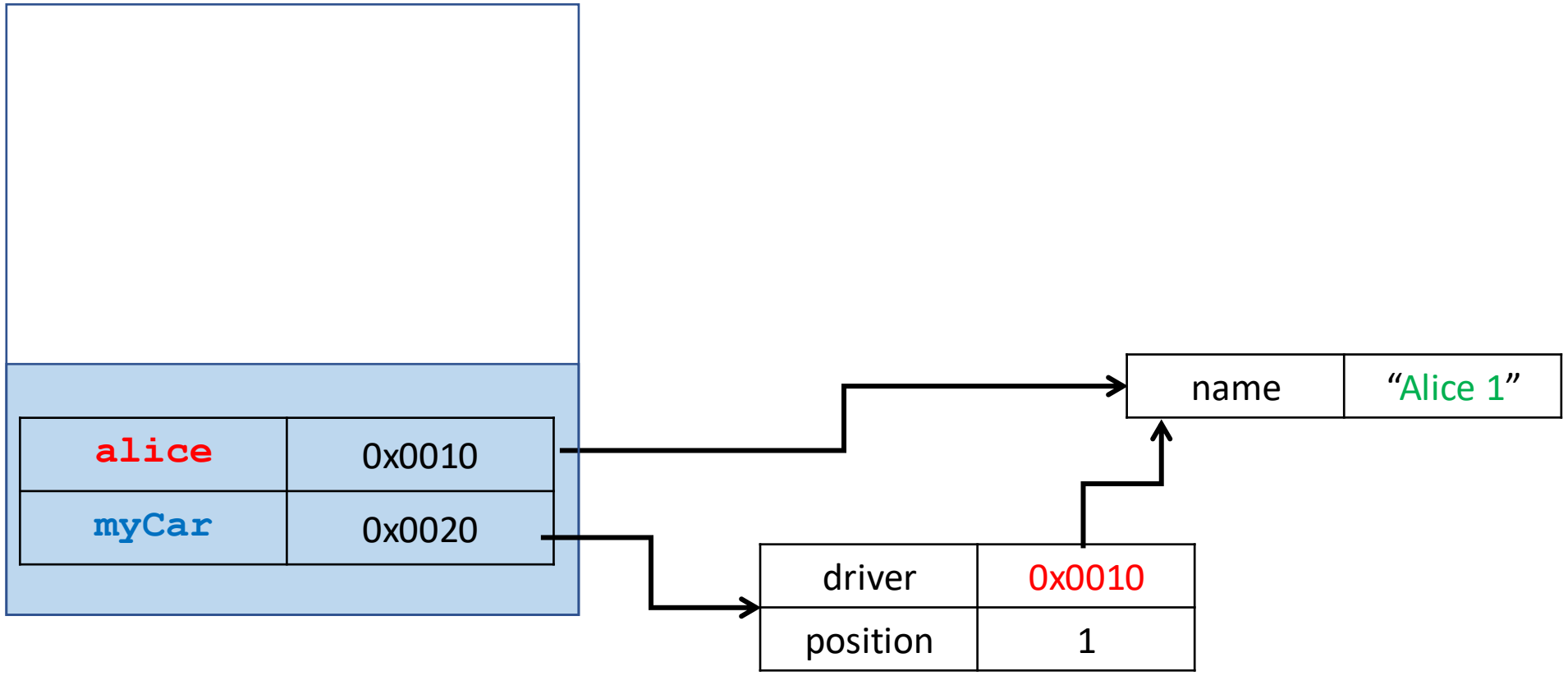
  public Car(int position, Person driver){
    this.position = position;
    this.driver = driver;
  }
  public Person getDriver(){
    return driver;
  }
}
```

```
class MovingCarDriver3
{
  public static void main(String args[]){
    Person alice = new Person("Alice 1");
    Car myCar = new Car(1, alice);
    alice.setName("Alice 2");
    System.out.println(myCar.getDriver().getName());
    alice = new Person("Alice 3");
    System.out.println(myCar.getDriver().getName());
  }
}
```

Τι θα τυπώσει?

Εκτέλεση

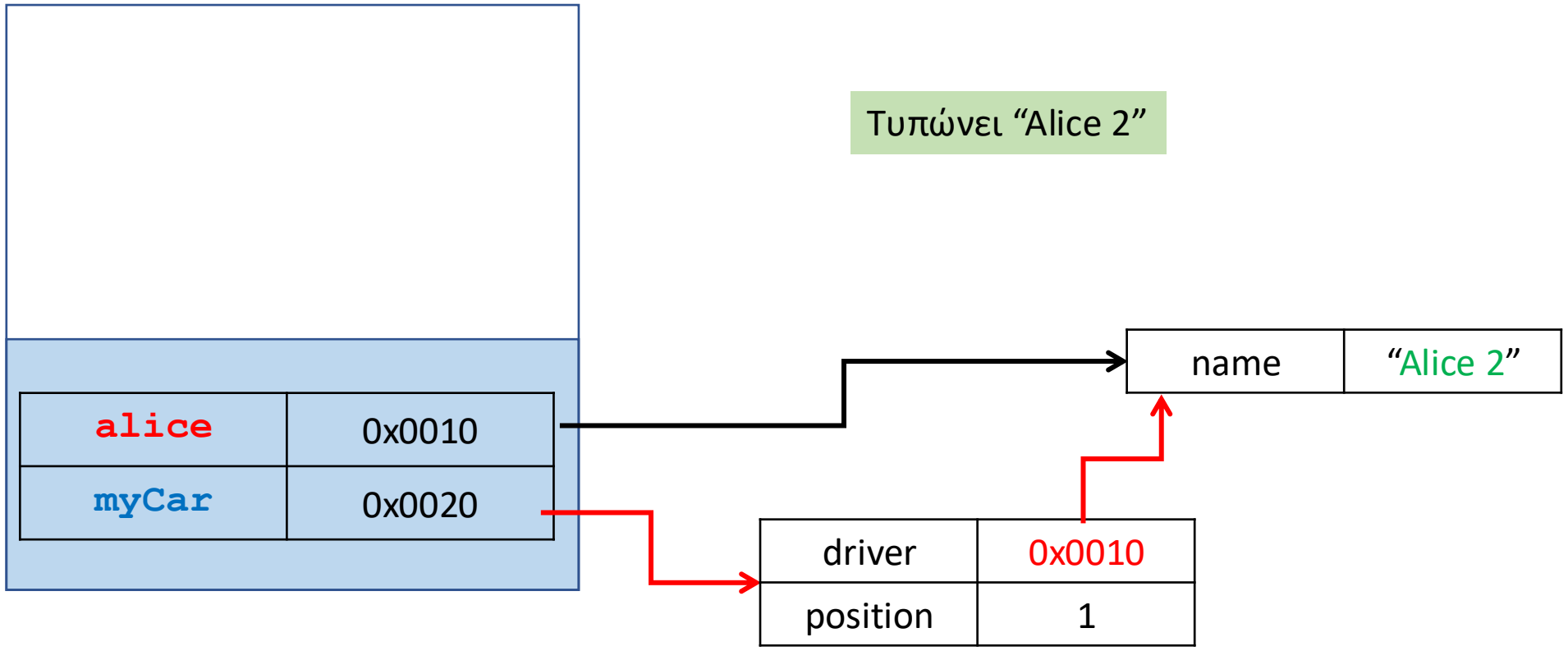
```
Person alice = new Person("Alice 1");
Car myCar = new Car(1, alice);
```



Εκτέλεση

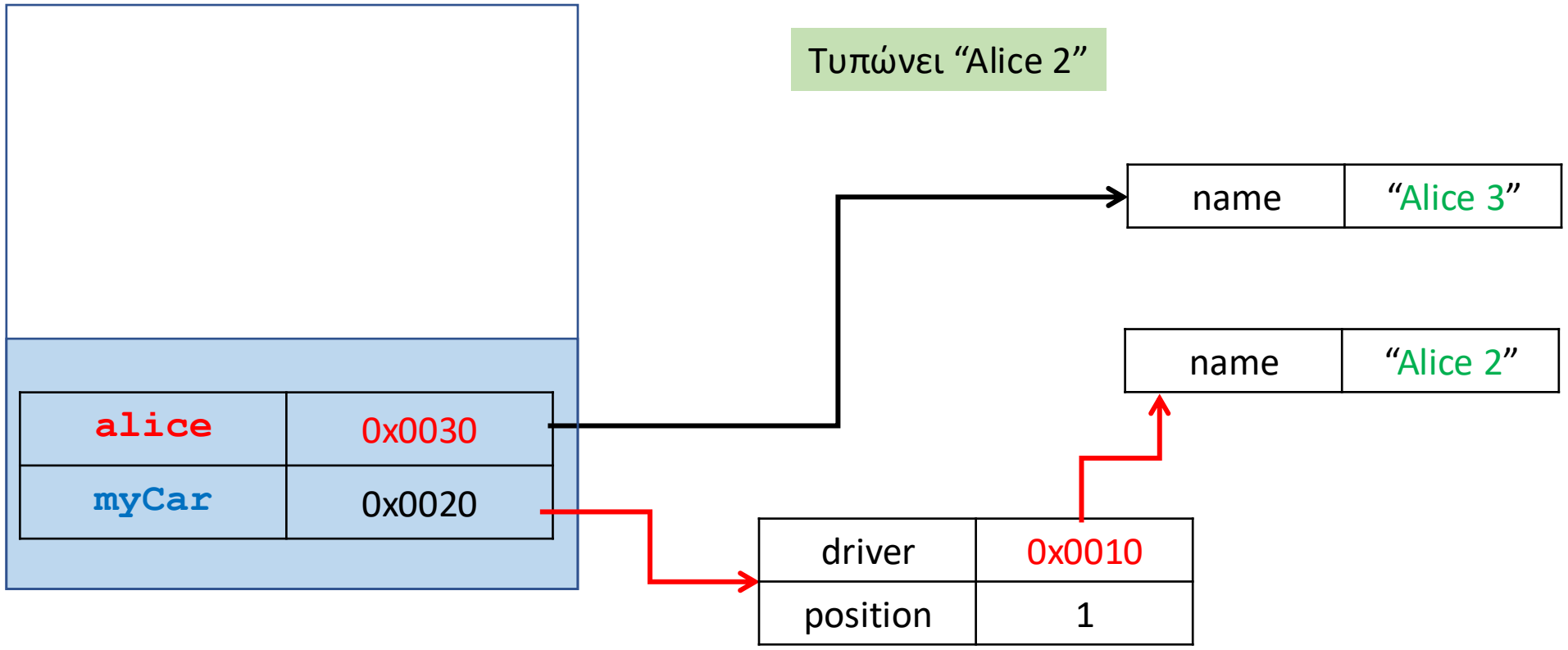
```
alice.setName("Alice 2");
System.out.println(myCar.getDriver().getName());
```

Τυπώνει “Alice 2”



Εκτέλεση

```
alice = new Person("Alice 3");
System.out.println(myCar.getDriver().getName());
```

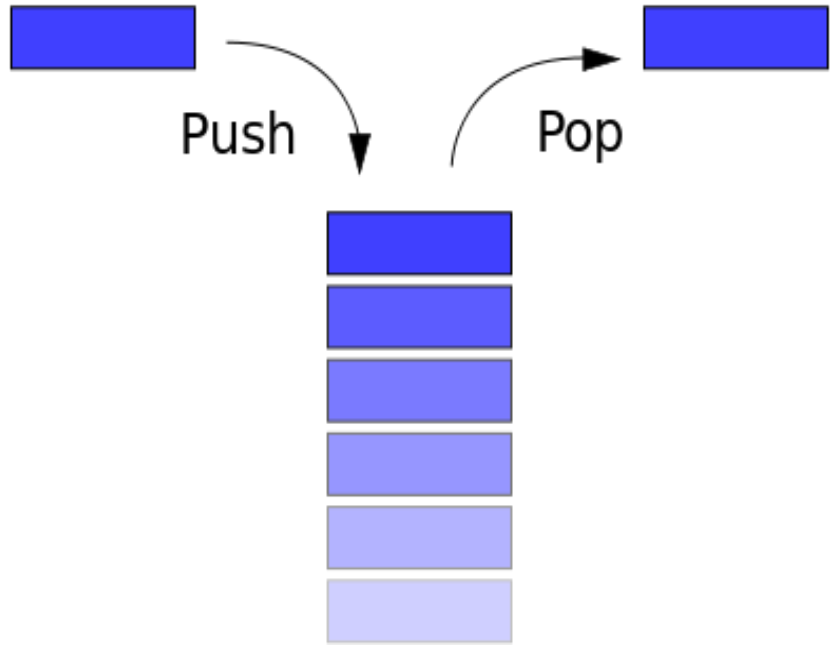


Αντικείμενα μέσα σε αντικείμενα

- Ορίζουμε κλάσεις για να ορίσουμε **τύπους δεδομένων** τους οποίους χρειαζόμαστε
  - Π.χ., ο τύπος δεδομένων Person για να μπορούμε να χειριζόμαστε πληροφορίες για ένα άτομο, και ο τύπος δεδομένων Car που κρατάει πληροφορία για το αυτοκίνητο
- Τους τύπους δεδομένων που ορίζουμε τους χρησιμοποιούμε για να δημιουργήσουμε μεταβλητές (αντικείμενα)
- Τα αντικείμενα μπορεί να είναι πεδία άλλων κλάσεων
  - Π.χ., η κλάση Car έχει ένα πεδίο τύπου Person
- Μία κλάση χρησιμοποιεί αντικείμενα άλλων κλάσεων και έτσι **συνθέτουμε** πιο περίπλοκους τύπους δεδομένων

Παράδειγμα: Στοίβα (Stack) - Υλοποίηση

- Η Στοίβα είναι μια συλλογή δεδομένων η οποία επιτρέπει τις εξής λειτουργίες:
  - **push(element)**: προσθέτει ένα νέο στοιχείο στην κορυφή της στοίβας
  - **pop()**: αφαιρεί και επιστρέφει το στοιχείο το οποίο βρίσκεται στην κορυφή της στοίβας
  - **isEmpty()**: ελέγχει αν η στοίβα είναι άδεια και επιστρέφει true ή false
- Η Στοίβα υλοποιεί την πολιτική Last-In-First-Out (LIFO) στη σειρά που μας δίνει τα στοιχεία
  - Χρήσιμο σε διάφορες εφαρμογές, π.χ., για τη δέσμευση μνήμης στην κλήση συναρτήσεων



Υλοποίηση

- Θα υλοποιήσουμε μια Στοίβα ακεραίων χρησιμοποιώντας ένα **πίνακα** (Στοίβα συγκεκριμένης χωρητικότητας)
  - Τι πεδία πρέπει να ορίσουμε?
  - Τι μεθόδους?

```
class Stack
{
    private int capacity;
    private int size = 0;
    private int[] elements;

    public Stack(int capacity){
        this.capacity = capacity;
        elements = new int[capacity];
    }

    public void push(int element){
        if (size == capacity){
            System.out.println("Cannot enter any more elements");
            return;
        }
        elements[size] = element;
        size ++;
    }

    public int pop(){
        if (size == 0){
            System.out.println("No elements to pop");
            return -1;
        }
        size-- ;
        return elements[size];
    }

    public boolean isEmpty(){
        return (size == 0);
    }
}
```

Εφαρμογές

- Υπολόγισε την δυαδική μορφή ενός ακεραίου

```
class Binary
{
	public static void main(String[] args)
	{
		Stack myStack = new Stack(100);
		int number = 1973;

		while (number > 0){
			myStack.push(number%2);
			number = number/2;
		}

		while (!myStack.isEmpty()){
			System.out.print(myStack.pop());
		}
	}
}
```

Επεκτάσεις

- Πως θα ορίσουμε την μέθοδο equals?
- Πως θα ορίσουμε τη μέθοδο toString?

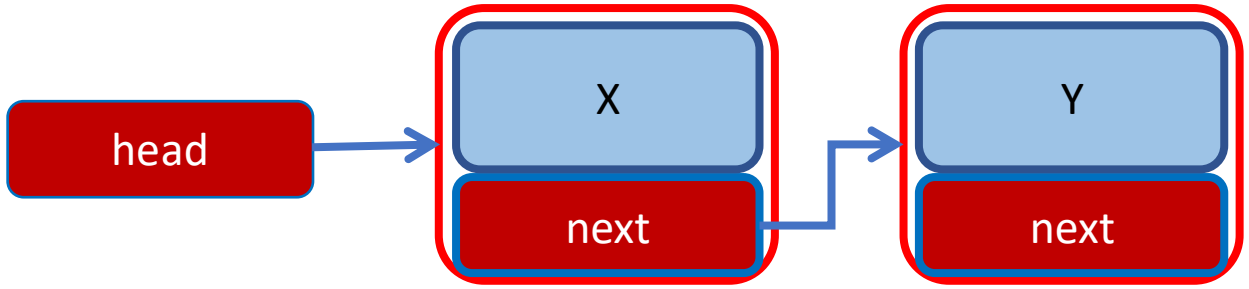
```
public String toString(){
        String returnString = "":
        for (int i = 0; i < size; i ++){
                returnString = returnString + elements[i] + " ";
        }
        return returnString;
}

public boolean equals(Stack other){
        if (this.size != other.size){
                return false;
        }
        for (int i = 0; i < size; i ++){
                if (this.elements[i] != other.elements[i]){
                        return false;
                }
        }
        return true;
}
```

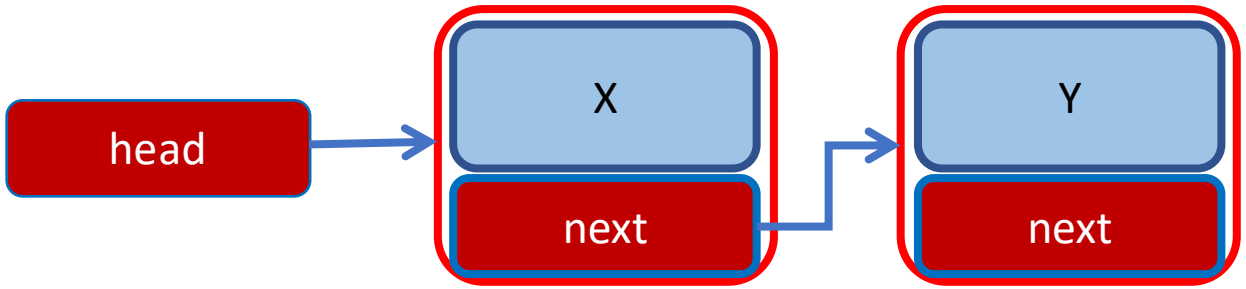
Είναι πιο εύκολο να ελέγξω τις προπτώσεις που είναι **false**, αν δεν ισχύει καμιά από αυτές τότε είναι **true**

Παράδειγμα

- Θέλουμε να υλοποιήσουμε την στοίβα ώστε να μην έχει περιορισμό στο μέγεθος (capacity)
- Βασική ιδέα:
  - Δημιουργούμε στοιχεία της στοίβας και τα συνδέουμε το ένα να δείχνει στο άλλο
  - Χρειάζεται να ξέρουμε και την κορυφή της στοίβας

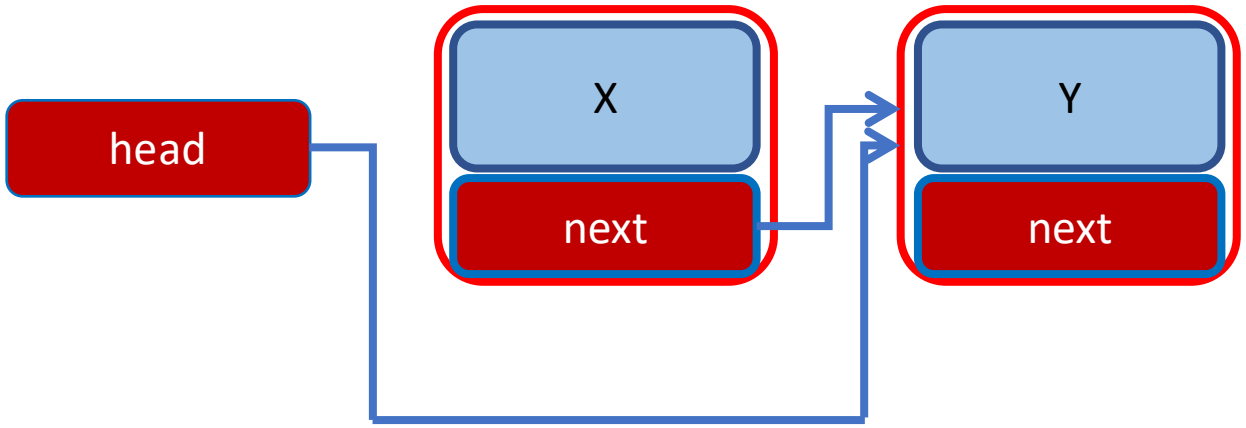


Στοίβα



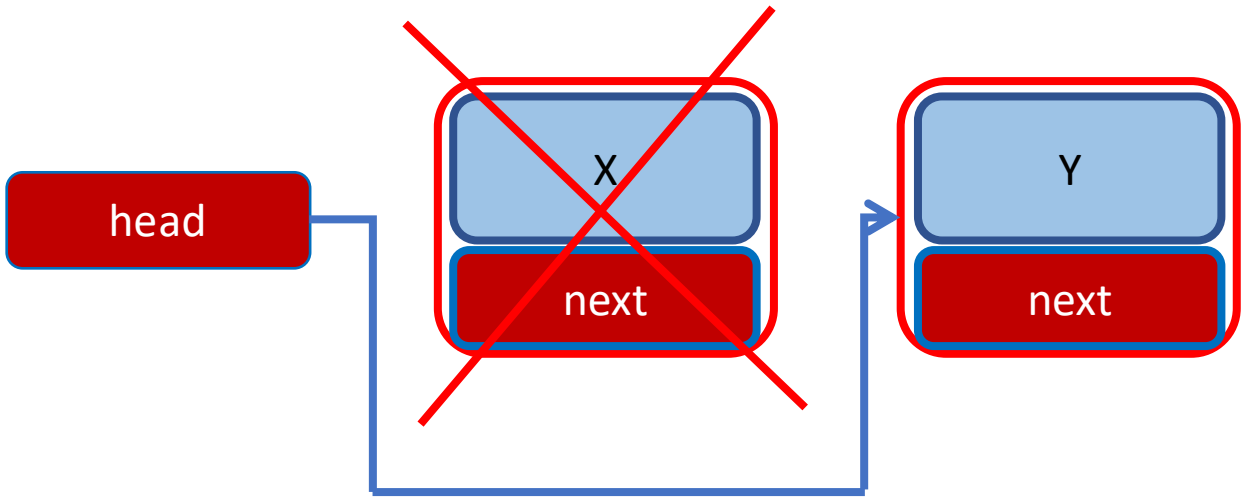
Pop(): Αφαιρεί το στοιχείο στην κορυφή της στοίβας και επιστρέφει την τιμή του (X στο παράδειγμα μας)

Στοίβα



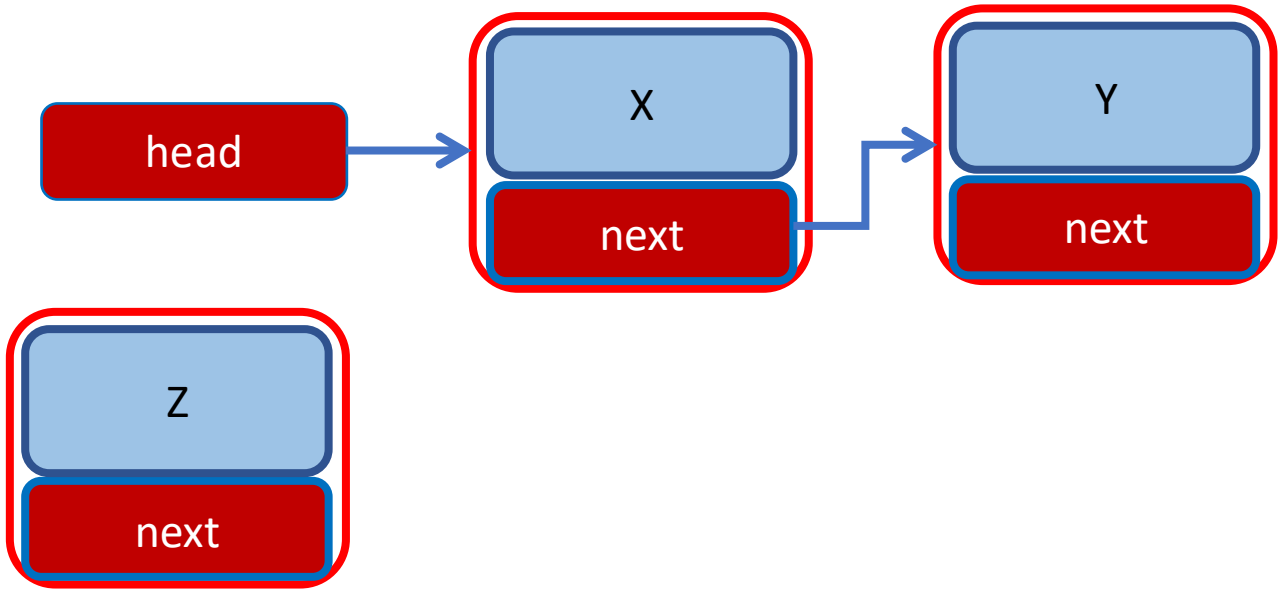
Pop(): Αφαιρεί το στοιχείο στην κορυφή της στοίβας και επιστρέφει την τιμή του (X στο παράδειγμα μας)

Στοίβα



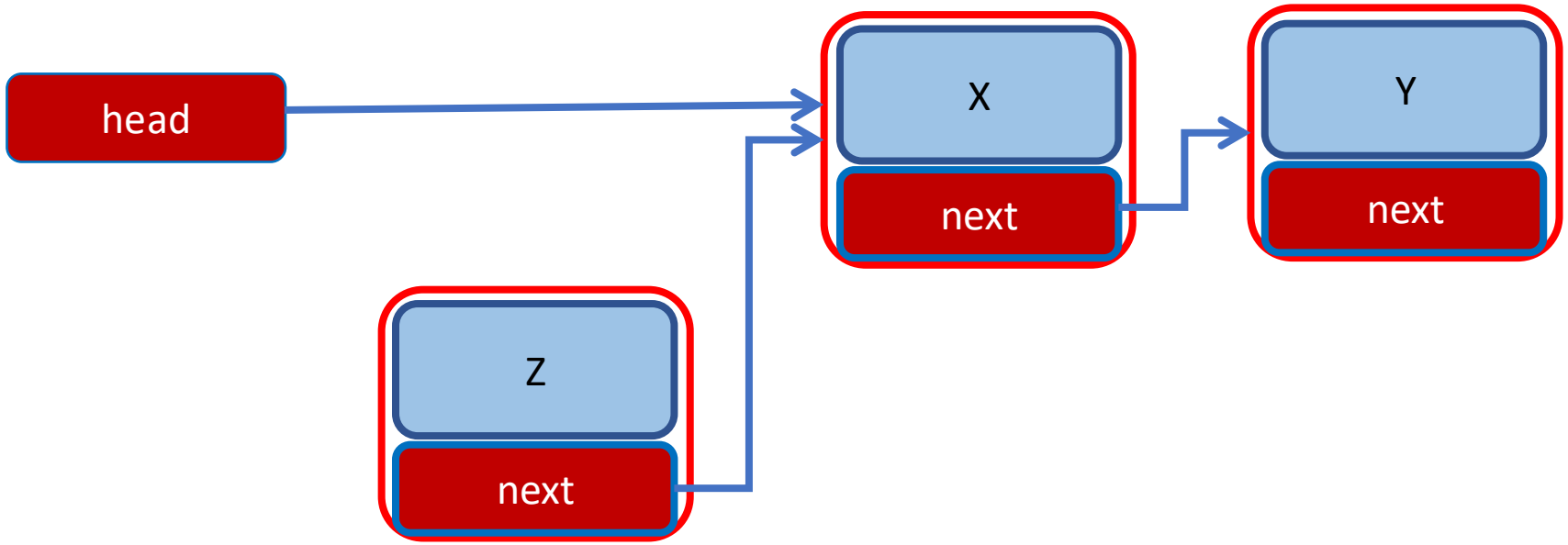
Pop(): Αφαιρεί το στοιχείο στην κορυφή της στοίβας και επιστρέφει την τιμή του (X στο παράδειγμα μας)

Στοίβα



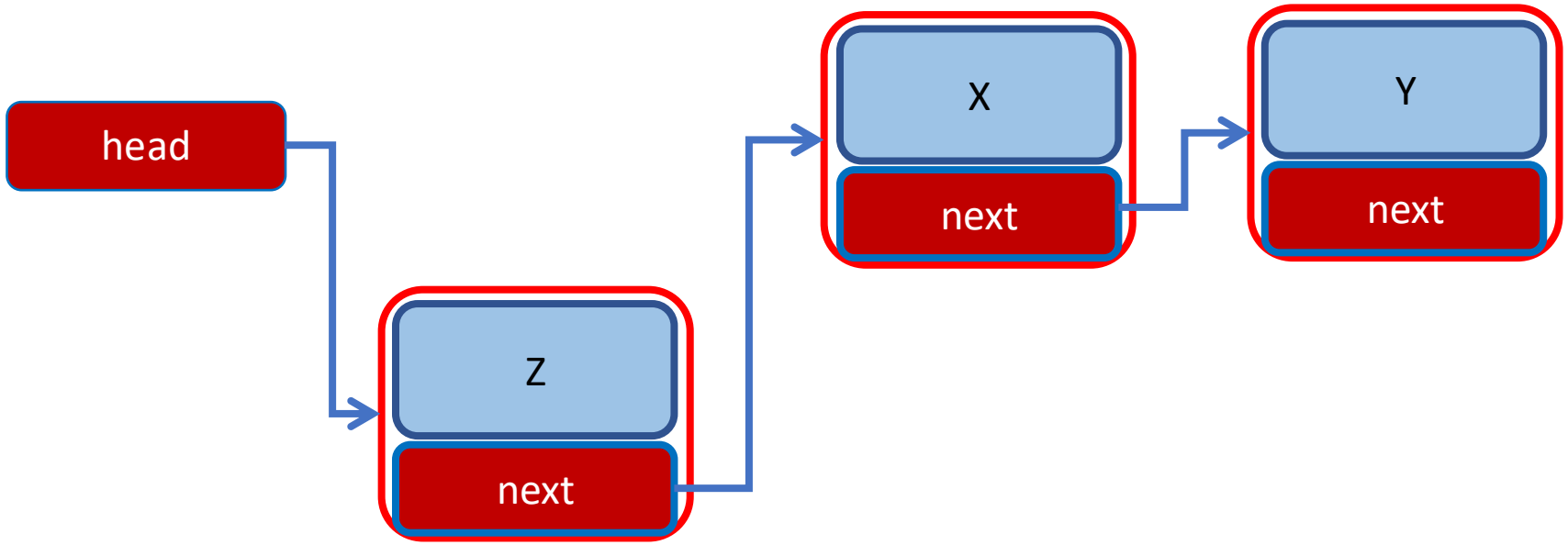
Push(Z): Προσθέτει την τιμή Z στην κορυφή της στοίβας

Στοίβα



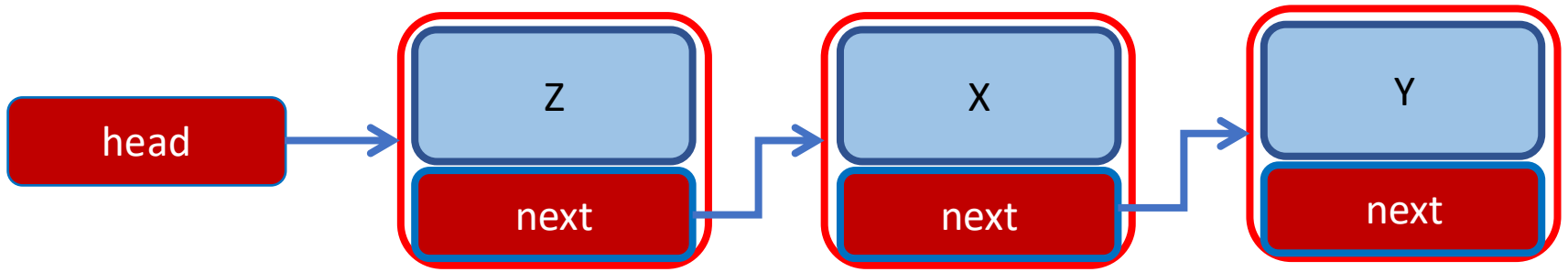
Push(Z): Προσθέτει την τιμή Z στην κορυφή της στοίβας

Στοίβα



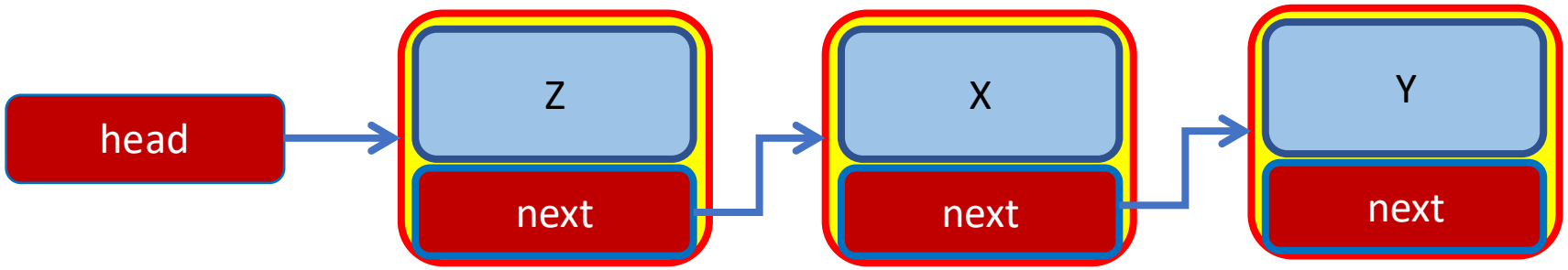
Push(Z): Προσθέτει την τιμή Z στην κορυφή της στοίβας

Στοίβα



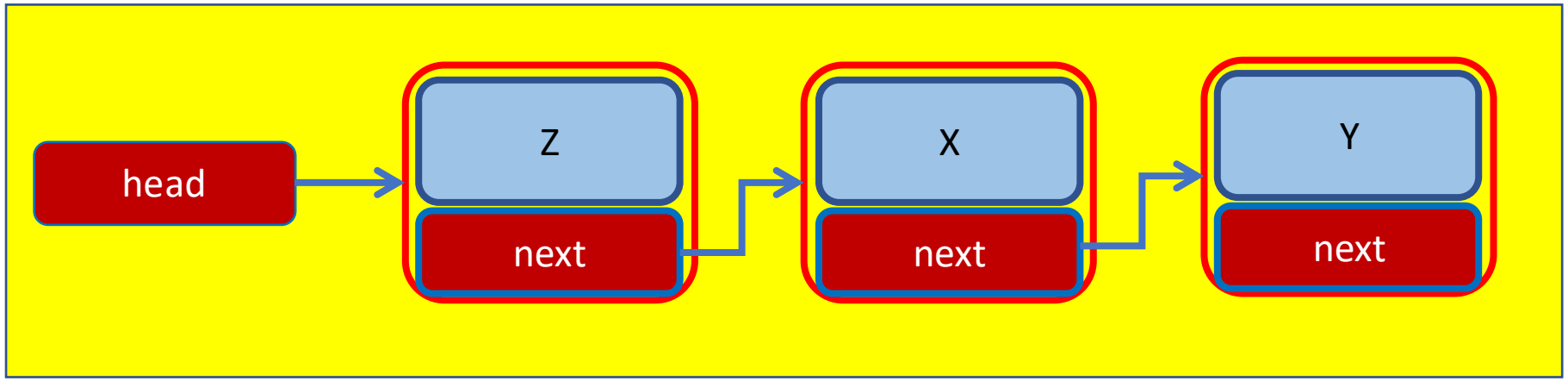
Push(Z): Προσθέτει την τιμή Z στην κορυφή της στοίβας

Στοίβα - Υλοποίηση



- Θα ορίσουμε StackElement μια κλάση που κρατάει το κάθε στοιχείο της στοίβας

Στοίβα - Υλοποίηση



- Θα ορίσουμε **StackElement** μια κλάση που κρατάει το κάθε στοιχείο της στοίβας
- Και μια κλάση **Stack** που υλοποιεί την στοίβα και όλες τις λειτουργίες της

```
class StackElement
{
        private int value;
        private StackElement next = null;

        public StackElement(int value){
                this.value = value;
        }

        public int getValue(){
                return value;
        }

        public StackElement getNext(){
                return next;
        }

        public void setNext(StackElement element){
                next = element;
        }

}
```

Το επόμενο στοιχείο

Επιστρέφει αντικείμενο

```
class Stack
{
        private StackElement head;
        private int size = 0;

        public int pop(){
                if (size == 0){ // head == null
                        System.out.println("Pop from empty stack");
                        System.exit(-1);
                }
                int value = head.getValue();
                head = head.getNext();
                size --;
                return value;
        }

        public void push(int value){
                StackElement element = new StackElement(value);
                element.setNext(head);
                head = element;
                size ++;
        }

}
```

Το πρώτο στοιχείο της στοίβας μας φτάνει για τα βρούμε όλα

Σταματάει την εκτέλεση του προγράμματος

Τα αντικείμενα τύπου StackElement δημιουργούνται μέσα στην Stack

Μέθοδος toString()

Χρειαζόμαστε μία StackElement μεταβλητή για να διατρέξει τα στοιχεία της στοίβας

```
public String toString(){
    String returnString = "";
    StackElement e = head;
    for (int i = 0; i < size; i ++){
       returnString = returnString + e.getValue() + " ";
       e = e.getNext();
    }
    return returnString;
}
```

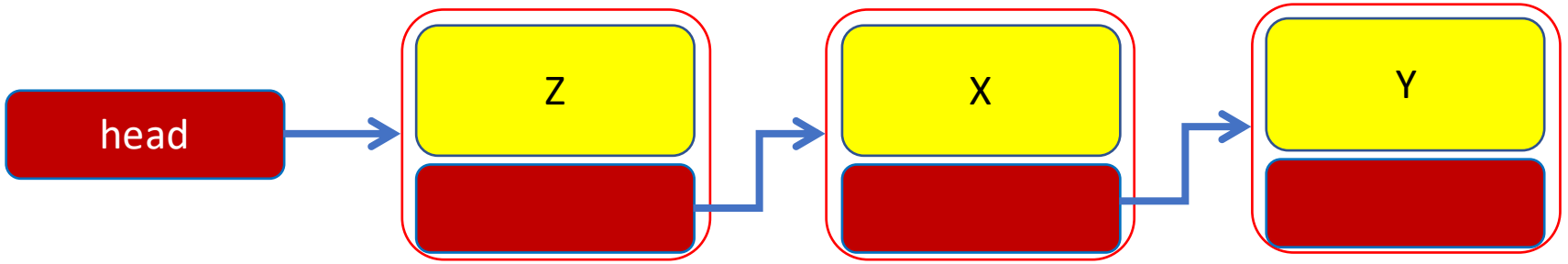
Εναλλακτική υλοποίηση

for-loop που δεν διατρέχει ακεραίους

```
public String toString(){
    String returnString = "";
    for (IntStackElement e = head; e != null; e = e.getNext()){
       returnString = returnString + e.getValue() + " ";
    }
    return returnString;
}
```

```
class StackExample
{
	public static void main(String[] args){
		Stack s = new Stack();
		s.push(3);
		s.push(2);
		s.push(1);
		System.out.println(s.pop());
		System.out.println(s.pop());
		System.out.println(s.pop());
		System.out.println(s.pop());
	}
}
```

Στοίβα - Υλοποίηση



- Τα X,Y,Z μπορεί να είναι δεδομένα οποιουδήποτε τύπου ή κλάσης
- Π.χ. αντί για ακέραιους θα μπορούσαμε να έχουμε αντικείμενα τύπου Person

```
class Person
{
	private String name;
	private int number;

	public Person(String name, int num){
		this.name = name;
		this.number = num;
	}

	public String toString(){
		return name+":"+number;
	}
}
```

```
class PersonStackElement
{
        private Person value;
        private PersonStackElement next;

        public PersonStackElement(Person val){
                value = val;
        }

        public void setNext(PersonStackElement element){
                next = element;
        }

        public PersonStackElement getNext(){
                return next;
        }

        public Person getValue(){
                return value;
        }

}
```

Ο constructor παίρνει σαν όρισμα το αντικείμενο που έχει ήδη δημιουργηθεί

Το αντικείμενο το χειριζόμαστε σαν μια οποιαδήποτε μεταβλητή

```
class Stack
{
        private PersonStackElement head;
        private int size = 0;

        public Person pop(){
                if (size == 0){ // head == null
                        System.out.println("Pop from empty stack");
                        return null;
                }
                int value = head.getValue();
                head = head.getNext();
                size --;
                return value;
        }

        public void push(Person value){
                StackElement element = new StackElement(value);
                element.setNext(head);
                head = element;
                size ++;
        }

}
```

Η pop πλέον επιστρέφει μεταβλητή τύπου Person

Επιστρέφουμε null για να σηματοδοτήσουμε ότι έγινε λάθος (όχι απαραίτητα ο καλύτερος τρόπος να το κάνουμε αυτό)

```
class StackExample
{
        public static void main(String[] args){
                PersonStack stack = new PersonStack();
                Person alice = new Person("Alice", 1);
                stack.push(alice);
                Person bob = new Person("Bob",2);
                stack.push(bob);
                Person charlie = new Person("Charlie",3);
                stack.push(charlie);
                System.out.println(stack.pop());
                System.out.println(stack.pop());
                System.out.println(stack.pop());
        }
}
```

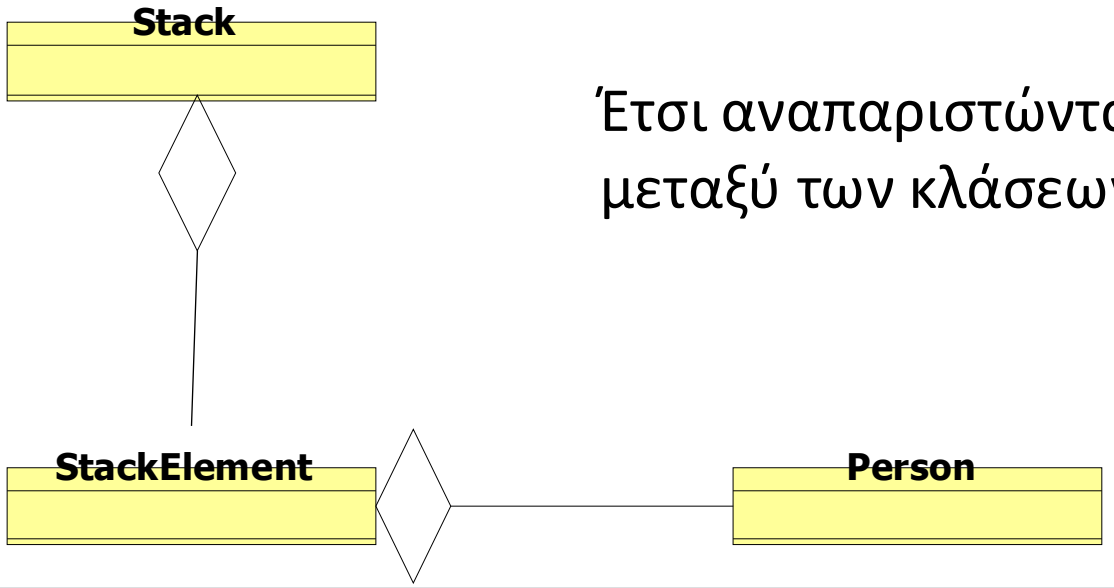
Προσοχή! Αν καλέσουμε άλλη μια φορά την pop θα πάρουμε runtime error γιατί προσπαθούμε να προσπελάσουμε null αναφορά

Σχέσεις μεταξύ κλάσεων

- Στο παράδειγμα με τη στοίβα έχουμε τρείς διαφορετικές κλάσεις (**Person, Stack StackElement**) οι οποίες συσχετίζονται μεταξύ τους με διαφορετικούς τρόπους
- Μπορεί να υπάρχουν πολλές διαφορετικές σχέσεις μεταξύ κλάσεων
  - Στην περίπτωση μας, η μία κλάση ορίζεται χρησιμοποιώντας αντικείμενα της άλλης

Η UML γλώσσα

- Η **UML** (**Unified Modeling Language**) είναι μια γλώσσα για να περιγράφουμε και να καταλαβαίνουμε τον κώδικα μας.
- Τα **UML διαγράμματα** δείχνουν τις σχέσεις μεταξύ των κλάσεων.



Έτσι αναπαριστώνται οι σχέσεις μεταξύ των κλάσεων

Σχέση συνάθροισης – Aggregation

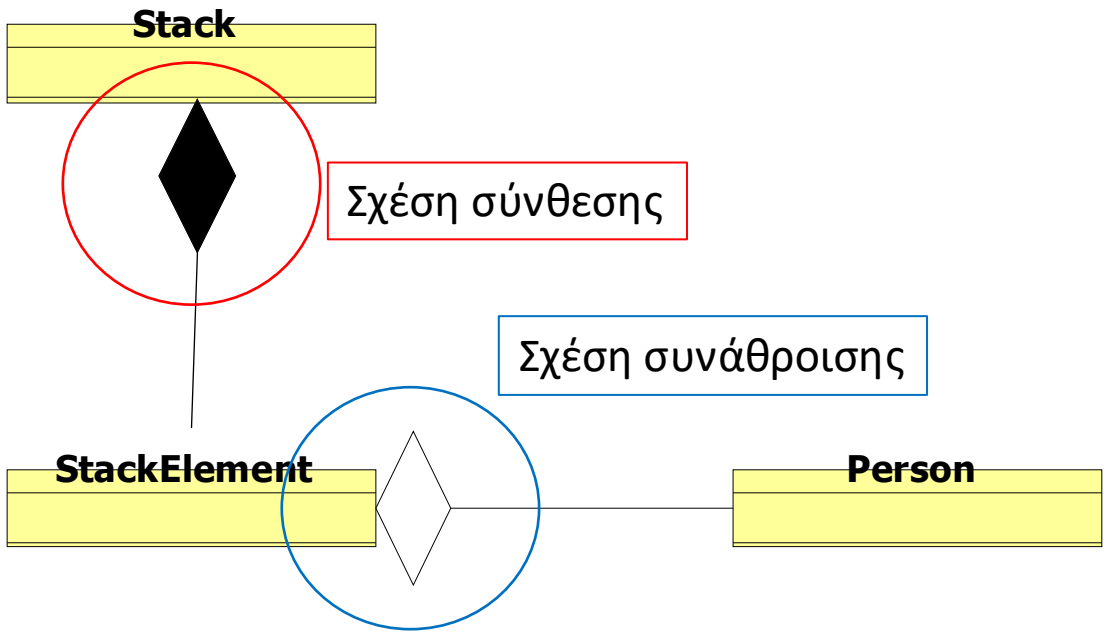
- Η κλάση **X** έχει σχέση συνάθροισης με την κλάση **Y**, αν αντικείμενο/α της κλάσης **Y** ανήκουν στο αντικείμενο της κλάσης **X**
- Τα αντικείμενα της κλάσης **Y** έχουν υπόσταση και εκτός της κλάσης **X**
  - Όταν καταστρέφεται ένα αντικείμενο της κλάσης **X** δεν καταστρέφονται απαραίτητα και τα αντικείμενα της κλάσης **Y**
- Παραδείγματα:
  - Σε έναν άνθρωπο μπορεί να ανήκει ένα αυτοκίνητο, ρούχα, κλπ.
  - Ένα κτήριο μπορεί να έχει μέσα ανθρώπους, έπιπλα, κλπ.
- Στην περίπτωση μας η κλάση StackElement έχει σχέση συνάθροισης με την κλάση Person

Σχέση σύνθεσης – Composition

- Η κλάση **X** έχει σχέση σύνθεσης με την κλάση **Y**, αν το αντικείμενο της κλάσης **X** **αποτελείται από** αντικείμενα της κλάσης **Y**
- Τα αντικείμενα της κλάσης **Y** **δεν υπάρχουν εκτός** της κλάσης **X**
  - Η κλάση **X** δημιουργεί τα αντικείμενα της κλάσης **Y**, και καταστρέφονται όταν καταστρέφεται το αντικείμενο της κλάσης **X**
- Παραδείγματα:
  - Ένας άνθρωπος αποτελείται από μέρη του σώματος: κεφάλι, πόδια, χέρια κλπ.
  - Ένα κτήριο αποτελείται από τοίχους, δωμάτια, πόρτες, κλπ.
- Στην περίπτωση μας η κλάση Stack έχει σχέση σύνθεσης με την κλάση StackElement

UML διαγράμματα

- Για να ξεχωρίζουν μεταξύ τους (κάποιες φορές) αναπαριστώνται διαφορετικά στα UML διαγράμματα



Προσοχή!

- Το σημαντικό είναι όταν δημιουργείτε το πρόγραμμα σας να σκεφτείτε ποιες κλάσεις χρειάζονται τα αντικείμενα που δημιουργούνται και πότε πρέπει να δημιουργηθούν μέσα στον κώδικα, και ποιες κλάσεις επηρεάζονται όταν αλλάζουν.

- Δεν υπάρχει χρυσός κανόνας. Γενικά το πώς θα σχεδιαστεί το πρόγραμμα είναι κάτι που μπορεί να γίνει με πολλούς τρόπους συνήθως. Διαλέξτε αυτόν που θα κάνει το πρόγραμμα πιο απλό, ευανάγνωστο, εύκολο να επεκταθεί, να ξαναχρησιμοποιηθεί και να διατηρηθεί.